

# Modelica 101 workshop

eSim conference 2026  
June 17, 2026

Massimo Cimmino, Ph.D.

Associate Professor

Department of Mechanical Engineering

Polytechnique Montréal

Kun Zhang, Ph.D.

Associate Professor

Department of Mechanical Engineering

École de technologie supérieure (ÉTS), Montréal





# Preamble

- Software configuration
  - Install OpenModelica
  - Download Modelica Buildings Library
  - Load Modelica Buildings Library in OpenModelica
  - Simulate an example model : Buildings.Examples.Tutorial.Boiler.System1
- Self-introduction
  - Name, program, university/organisation
  - Experience with Building Performance Simulation (BPS) tools (e.g., EnergyPlus, TRNSYS) and Simulink
  - Motivation for this workshop
- Schedule
  - Session 1: 09:00 – 10:30; Coffee break: 10:30 – 10:50
  - Session 2: 10:50 – 12:00; Lunch break: 12:00 – 13:00
  - Session 3: 13:00 – 14:30; Coffee break: 14:30 – 14:50
  - Session 4: 14:50 – 16:00



# Learning objectives

- Explain the Modelica language and its differences from other BPS tools
- Familiarize with Modelica Standard Library and Buildings Library
- Know Modelica projects and best practices
- Gain hands-on experience of model construction
- Develop a simple Modelica model using programming
- Visualize and analyze simulation results
- Build your own Modelica model
  
- Not covered
  - Modelica grammar, syntax and advanced functions



# Workshop plan

- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



# Workshop plan

- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



# Introduction to Modelica: Part I

## Modelica language: History

- Several equation-based **software** have been used for building simulation: ENET (1982), SPANK/SPARK (1986), IDA/ICE (language NMF, 1989), EES (1993).
- The Dymola **language** was envisioned by Elmqvist (1978) et later developed under the name **Modelica** (Mattson & Elmqvist, 1993).
  - Dymola is now a commercial software developed by Daussault Systèmes for the Modelica language. Other software exist, including the open-source [OpenModelica](#).
  - Modelica development is supported by international collaboration efforts through the non-profit [Modelica Association](#). The Modelica Language specification is open (CC BY-SA 4.0). The Modelica Association also develops the Modelica Standard Library and the Function Mockup Interface (FMI) standard.



# Introduction to Modelica: Part I

## Modelica language: Key features

- Modelica is an **object-oriented**, **equation-based** language for multi-domain (thermal, mechanical, electrical, control, etc.) modeling of dynamical systems.
- It has the following features:
  - Encapsulation;
  - Inheritance;
  - Hierarchical modelling.
- It supports component-based modeling to build models with fundamental blocks or compositions using generalized topological connections.





# Introduction to Modelica: Part I

## OpenModelica: Graphical user interface

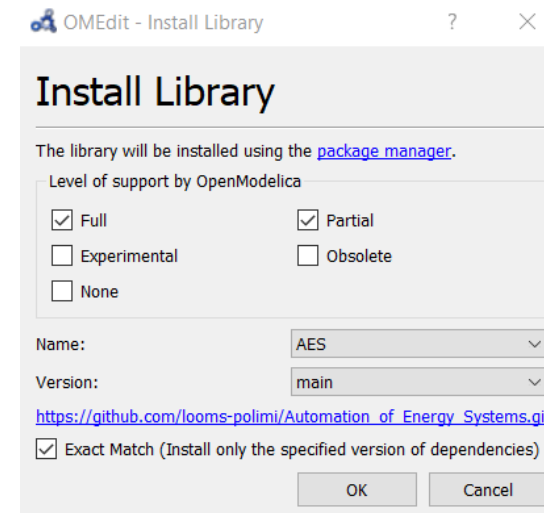
- Welcome perspective
  - Library management
- Modelling perspective
  - Buildings.Examples.Tutorial.Boiler.System1
  - Icon
  - Diagram
  - Text
  - Documentation: HTML syntax
- Plotting perspective
  - Plot space temperature
  - Tools, Open Working Directory
  - Change working directory: Tools, Options, General
  - Export variables: csv
- Debugging perspective



# Introduction to Modelica: Part I

## Modelica Libraries/Packages

- Information page 
- Runnable model  vs. non-runnable model
- Component path in a library
  - Example: Modelica.Blocks.Logical.And
- Search function
  - Example: FixedHeatFlow
- Package management
  - Installation
  - Loading: Tools, Options, Libraries
- Overview of Buildings Library





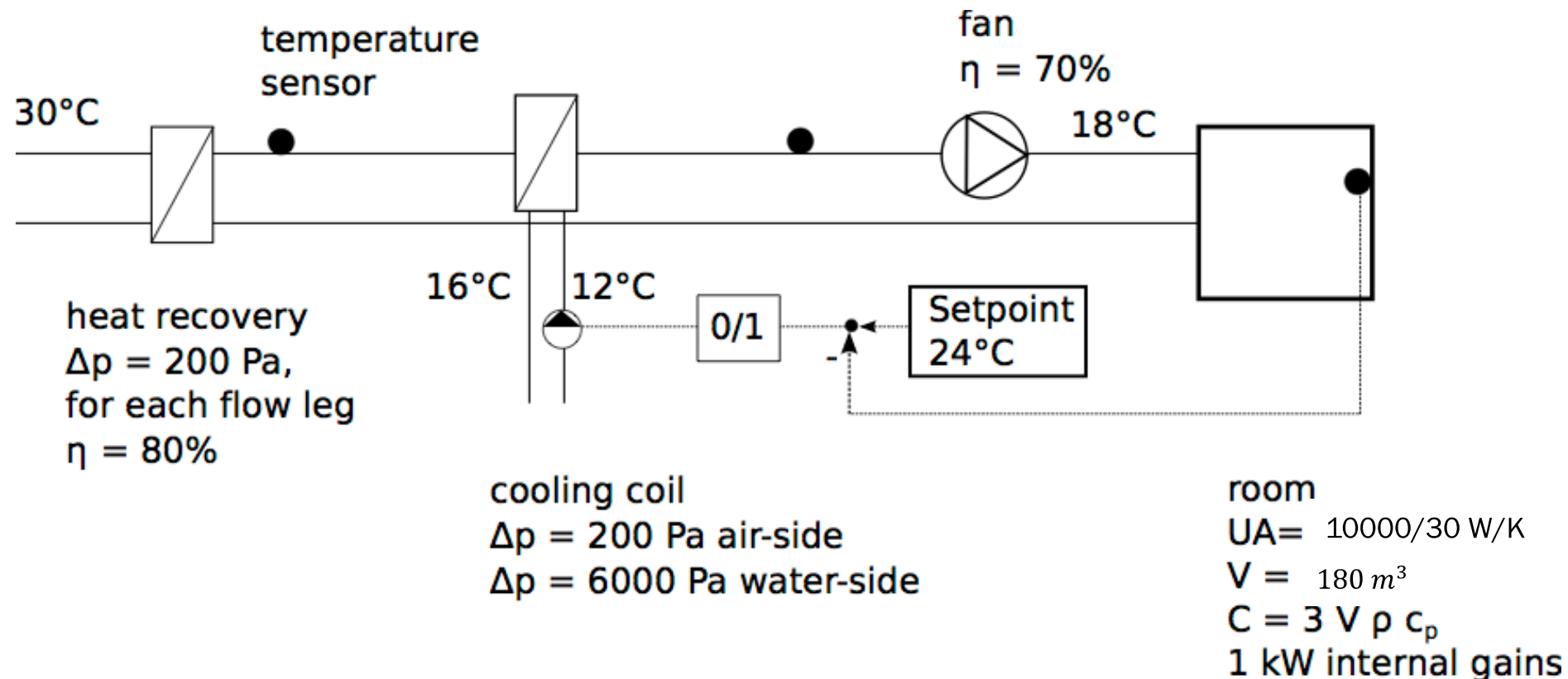
# Workshop plan

- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



# Model plan and workflow

- Tutorial Part I: Modelling a room with 1R1C (**Model 1**)
- Tutorial Part II: Modelling an Air Handling Unit (AHU) (Model 2)
- Tutorial Part III: Modelling on/off control of an AHU (Model 3)

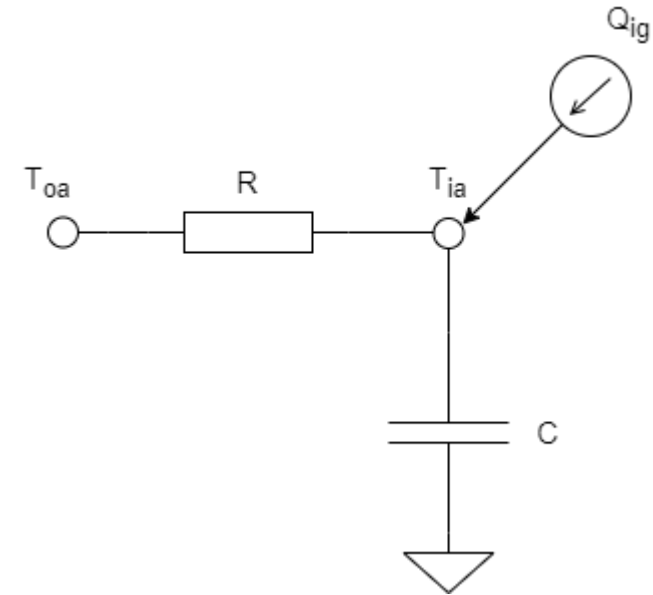




# Tutorial Part I: Equation-based modeling

## Model 1: Modeling a room with 1R1C

- Room parameters
  - Floor area:  $60 \text{ m}^2$
  - Height:  $3 \text{ m}$
  - Overall heat loss coefficient  $UA$ :  $\frac{10000}{30} \text{ W/K}$
  - Internal heat gains:  $1000 \text{ W}$
  - Initial temperature:  $20 \text{ }^\circ\text{C}$
- Boundary condition
  - Outdoor air temperature:  $30 \text{ }^\circ\text{C}$





# Tutorial Part I: Equation-based modeling

## Differential-algebraic system of equations (DAE)

- A generic differential-algebraic system of equations is given by:

$$F(t, \dot{x}(t), x(t), u(t), y(t), \Theta) = 0 \leftarrow \text{Rate of change}$$

$$Y(t, x(t), u(t), y(t), \Theta) = 0 \leftarrow \text{Algebraic constraints}$$

$$F_0(x(t_0), u(t_0), y(t_0), \Theta) = 0 \leftarrow \text{Initial conditions}$$

- where:

- $t$  : time
- $x$  : state variables
- $u$  : control signals and disturbances
- $y$  : algebraic variables
- $\Theta$  : parameters



# Tutorial Part I: Equation-based modeling

## Model 1: DAEs of a 1R1C model

- The DAEs of a 1R1C model are given by:

$$C \frac{dT_{ia}}{dt} = Q_{ig} + \frac{T_{oa} - T_{ia}}{R}$$

$$T_{ia}(0) = T_{ia,0}$$

- where:

- $C = 60 \text{ m}^2 \cdot 3 \text{ m} \cdot \frac{1.2 \text{ kg}}{\text{m}^3} \cdot \frac{1 \text{ kJ}}{\text{kg} \cdot \text{K}}$

- $R = \frac{1}{UA} = \frac{30}{10000} \frac{\text{K}}{\text{W}}$

- $Q_{ig} = 1000 \text{ W}$

- $T_{oa} = 30 + 273.15 \text{ K}$

- $T_{ia,0} = 20 + 273.15 \text{ K}$



# Tutorial Part I: Equation-based modeling

## Model 1: Modelica script

```
model Room_DAE "This is a model of a room, implemented using equation-based modeling"
  extends Modelica.Icons.Example;

  // Parameters (Paramètres)
  parameter Modelica.Units.SI.Area A_floor = 60 "Floor area of the room (Surface de plancher de la pièce)";
  parameter Modelica.Units.SI.Height h = 3 "Height of the room (Hauteur de la pièce)";
  parameter Modelica.Units.SI.ThermalConductance UA = 10000/30 "Overall heat loss coefficient (Coefficient de déperditions thermiques)";
  parameter Modelica.Units.SI.HeatFlowRate Q_ig = 1000 "Internal heat gains (Gains thermiques internes)";
  parameter Modelica.Units.SI.Temperature T_oa(displayUnit = "degC") = 30 + 273.15 "Outdoor air temperature (Température extérieure)";
  parameter Modelica.Units.SI.Density rho_air = 1.2 "Air density (Densité de l'air)";
  parameter Modelica.Units.SI.SpecificHeatCapacity cp_air = 1000 "Specific heat of air (Chaleur spécifique de l'air)";
  parameter Modelica.Units.SI.Temperature T_ia0(displayUnit = "degC") = 20 + 273.15 "Initial temperature (Température initiale)";

  // Lumped parameters (Paramètres intermédiaires)
  final parameter Modelica.Units.SI.HeatCapacity C = A_floor * h * rho_air * cp_air "Lumped capacitance of the room (Capacité thermique de la pièce)";
  final parameter Modelica.Units.SI.ThermalResistance R = 1 / UA "Thermal resistance to the outdoor (Résistance thermique vers l'extérieur)";

  // Variables
  Modelica.Units.SI.Temperature T_ia(displayUnit = "degC") "Temperature of the indoor air (Température intérieure)";

initial equation
  T_ia = T_ia0;

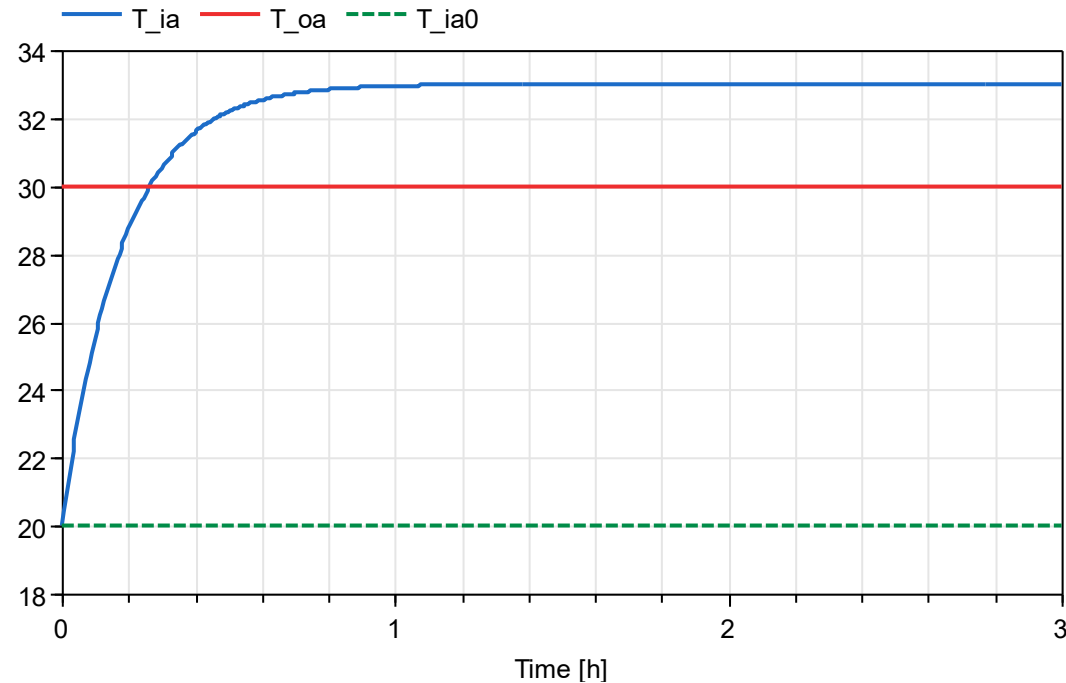
equation
  C * der(T_ia) = Q_ig + (T_oa - T_ia) / R;

annotation (
  experiment(StartTime = 0, StopTime = 10800, Interval = 20., Tolerance = 1e-06));
end Room_DAE;
```



# Tutorial Part I: Equation-based modeling

## Model 1: Results

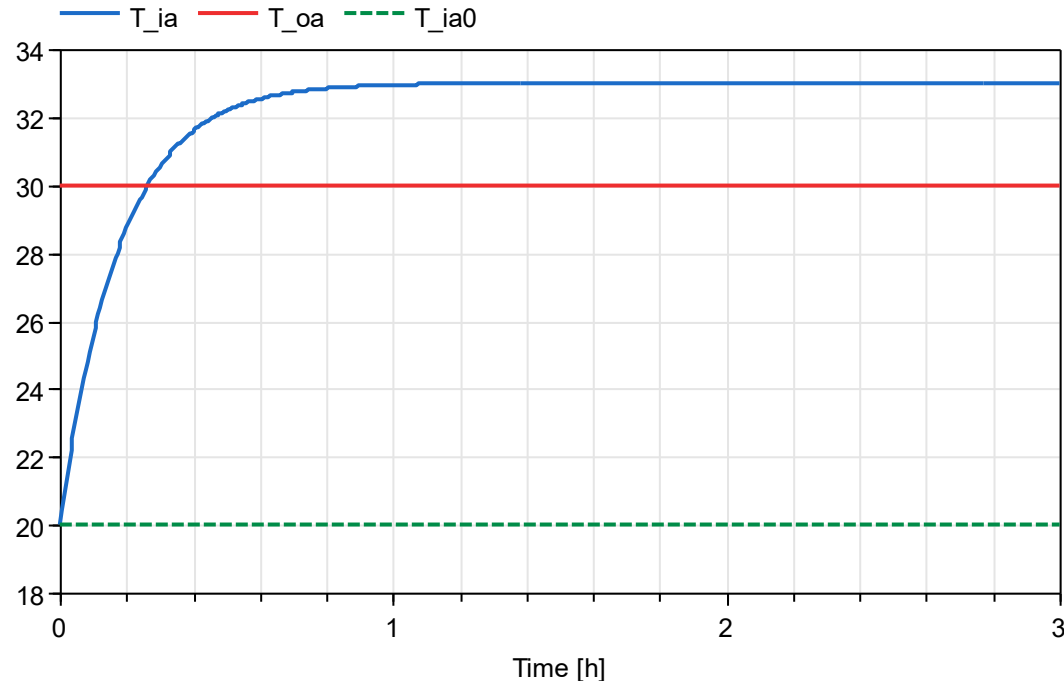


- Reflect on the results
  - What is the steady-state temperature? Is it correct?
- Try increasing the thermal capacity of the room:  
$$C = 3 * A_{\text{floor}} * h * \rho_{\text{air}} * c_{p,\text{air}}$$



# Tutorial Part I: Equation-based modeling

## Model 1: Steady-state analysis



- Governing equation:

$$C \frac{dT_{ia}}{dt} = Q_{ig} + \frac{T_{oa} - T_{ia}}{R}$$

- In steady state, the left term equals 0:

$$T_{ia} = T_{oa} + \frac{Q_{ig}}{R}$$

$$T_{ia} = 30 + \frac{30}{10000} \cdot 1000$$

$$T_{ia} = 33^{\circ}\text{C}$$

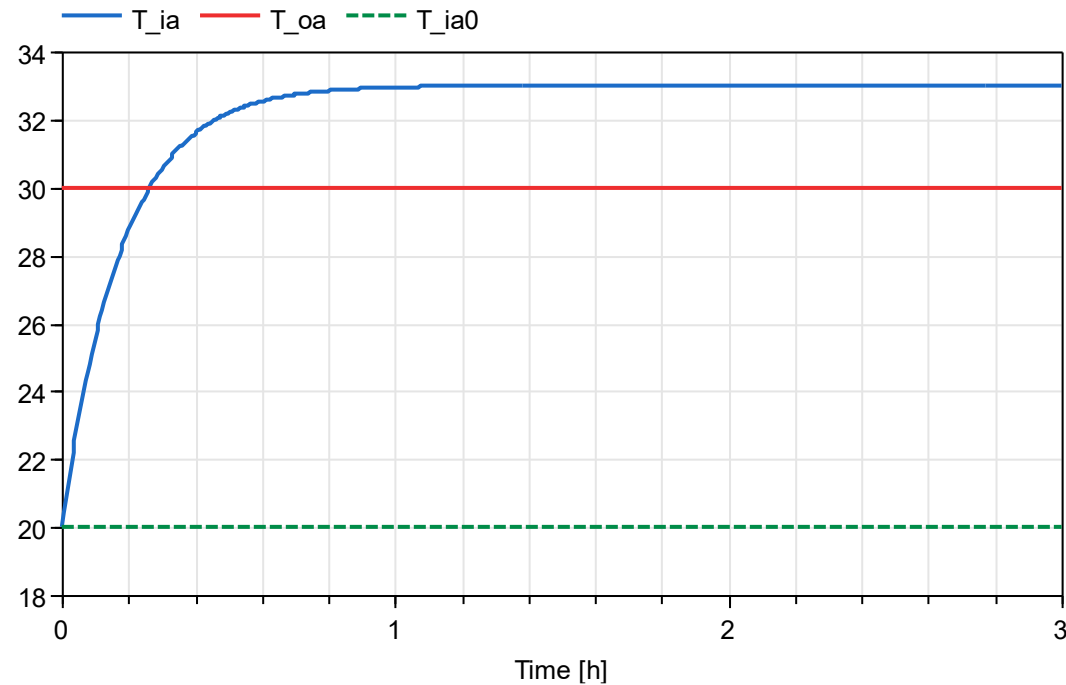


# Tutorial Part I: Equation-based modeling

## Model 1: Thermal capacity of the room

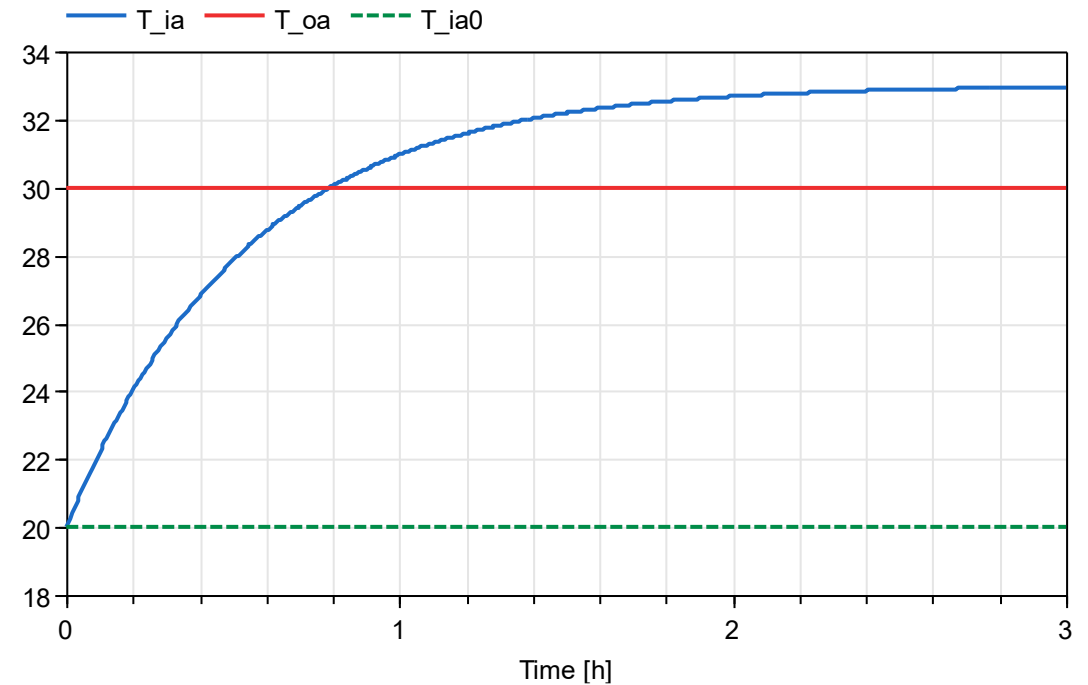
- Base thermal capacity

$$C = A_{\text{floor}} * h * \rho_{\text{air}} * c_{p_{\text{air}}}$$



- Tripled thermal capacity

$$C = 3 * A_{\text{floor}} * h * \rho_{\text{air}} * c_{p_{\text{air}}}$$





# Workshop plan

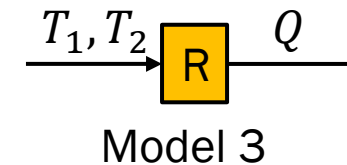
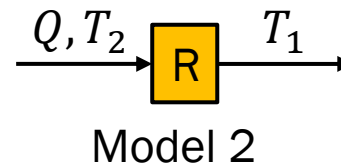
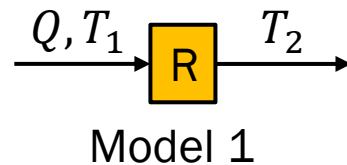
- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



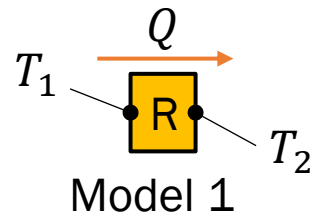
# Tutorial Part I: Component-based modeling

## Causal vs Acausal models

- A model is **causal** when the direction of the flow of information through the model is fixed (inputs and outputs are fixed).
  - For a thermal resistance :



- A model is **acausal** when the direction of the flow of information is not explicitly defined (boundaries are specified).
  - For a thermal resistance :



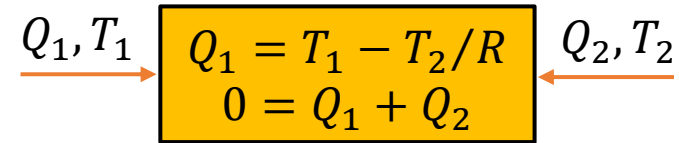
The solution is then possible if there are as many equations as unknowns to complete the DAE.



# Tutorial Part I: Component-based modeling

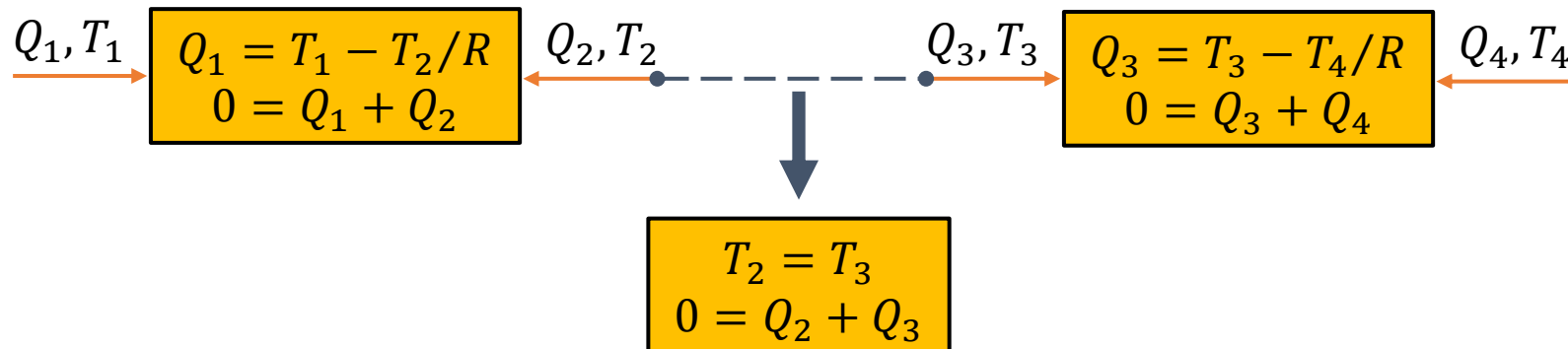
## Nodes

- We differentiate **potential** from **flow variables** (and also **stream variables**).



- When connecting 2 models:
  - Equality equations are introduced for potential variables
  - Conservation equations are introduced for flow variables
  - Mixing equations are introduced for stream variables

→ Connecting components is the same as adding equations.



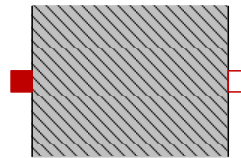


# Tutorial Part I: Component-based modeling

## Nodes (Continued)

- Nodes are an acausal generalization of inputs/outputs in causal models.
- A node encapsulates all information associated with a physical domain:
  - Thermal nodes are defined by the heat flow rate and the temperature ( $Q$  and  $T$ ).

thermalResistor



$R=R$

- Fluid nodes are defined by the mass flow rate, the pressure, the specific enthalpy and trace substance concentrations ( $\dot{m}$ ,  $p$ ,  $h$ ,  $x$ ).



pipe



# Tutorial Part I: Component-based modeling

## Model 1: Modeling a room with 1R1C

- Add components (search the component name)
  - Zone: Buildings.Fluid.MixingVolumes.MixingVolume (**zon**)
  - Heat loss: Modelica.Thermal.HeatTransfer.Components.ThermalConductor (**theCon**)
  - Internal heat gains: Modelica.Thermal.HeatTransfer.Sources.FixedHeatFlow (**intGains**)

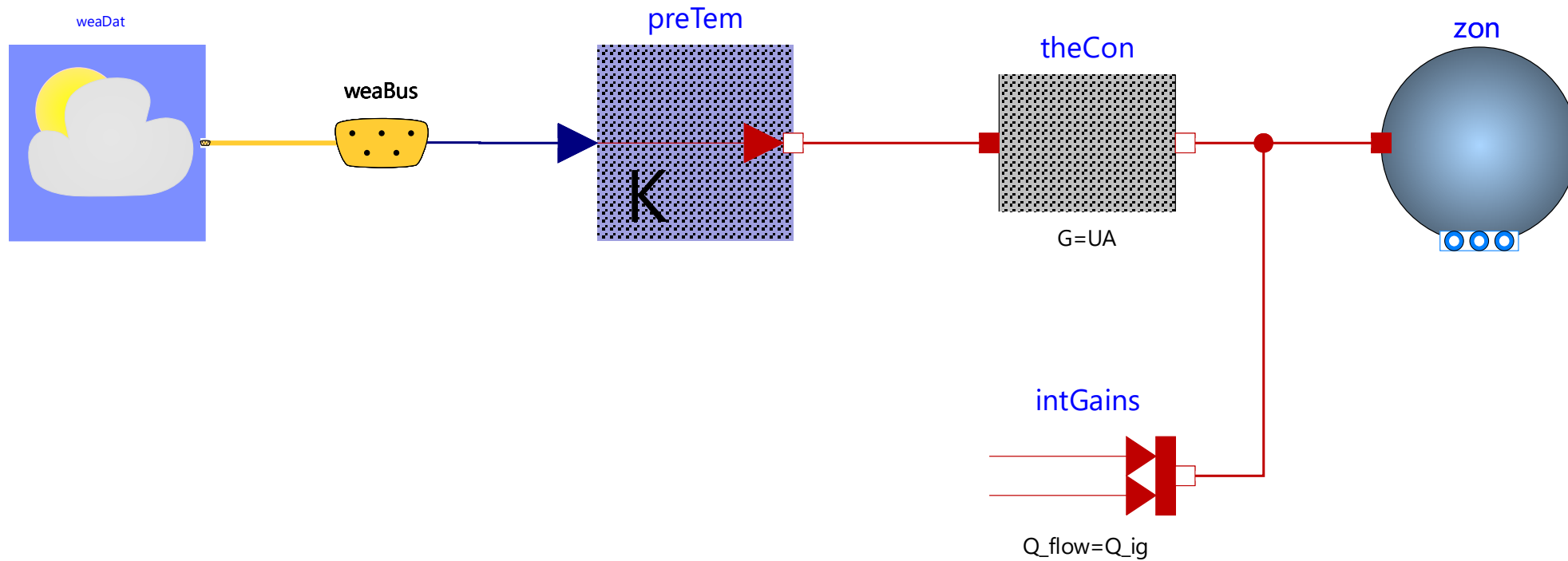
We will use the dry bulb temperature from a weather file instead of a fixed outdoor temperature:

- Outdoor temperature: Modelica.Thermal.HeatTransfer.Sources.PrescribedTemperature (**preTem**)
  - Weather: Buildings.BoundaryConditions.WeatherData.ReaderTMY3 (**weaDat**)
  - Weather bus: Buildings.BoundaryConditions.WeatherData.Bus (**weaBus**)
- Connect components



# Tutorial Part I: Component-based modeling

## Model 1: Modelica diagram





# Tutorial Part I: Component-based modeling

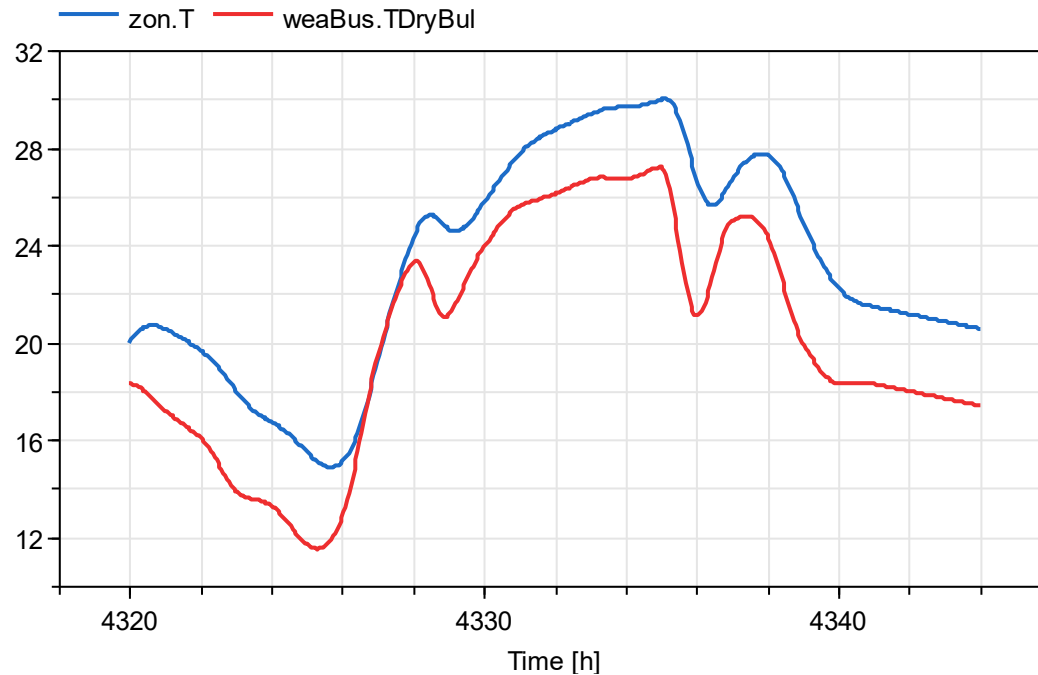
## Model 1: Model parameters

- Specify parameters
  - Room:
    - Under “General”:
      - Medium = Buildings.Media.Air (“Moist Air”)
      - $V = 180 \text{ m}^3$ ;  $m_{\text{flow\_nominal}} = 0.646 \text{ kg/s}$
    - Under “Dynamics”:
      - `energyDynamics = Modelica.Fluid.Types.Dynamics.FixedInitial`
      - `mSenFac = 3`
    - Under “Initialization”:
      - $T_{\text{start}} = 20 + 273.15 \text{ K}$
  - Heat loss:  $G = 10000/30 \text{ W/K}$
  - Internal heat gains:  $Q_{\text{flow}} = 1000 \text{ W}$
  - Weather: `FilNam = “USA_IL_Chicago-OHare.Intl.AP.725300_TMY3.mos”`
- Check model
- Simulation setup
  - Stop time:  $3h = 10800s$
- Documentation (HTML format)
  - <https://simulationresearch.lbl.gov/modelica/releases/v13.0.0/help/Buildings.html>



# Tutorial Part I: Component-based modeling

## Model 1: Results



- Using TMY data for outdoor temperature
  - Best practice: use function `loadResource` to load external files (path is relative to the Buildings library)
  - Absolute path
    - `C:/my_path/USA_IL_Chicago-OHare.Intl.AP.725300_TMY3.mos`
  - Relative path
    - `Modelica.Utilities.Files.loadResource("modelica://Buildings/Resources/weatherdata/USA_IL_Chicago-OHare.Intl.AP.725300_TMY3.mos")`
  - .mos file: weather format in Modelica
    - [https://energyplus.net/weather-location/north\\_and\\_central\\_america\\_wmo\\_region\\_4/USA/IL/USA\\_IL\\_Chicago-OHare.Intl.AP.725300\\_TMY3](https://energyplus.net/weather-location/north_and_central_america_wmo_region_4/USA/IL/USA_IL_Chicago-OHare.Intl.AP.725300_TMY3)
- Simulation setup:
  - Start time: 180 d (June 30) = 4320 h = 15552000 s
  - Stop time: 181 d = 4344 h = 15638400 s
  - Tick box: **save experiment annotation**



# Workshop plan

- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- **Introduction to Modelica: Part II**
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction

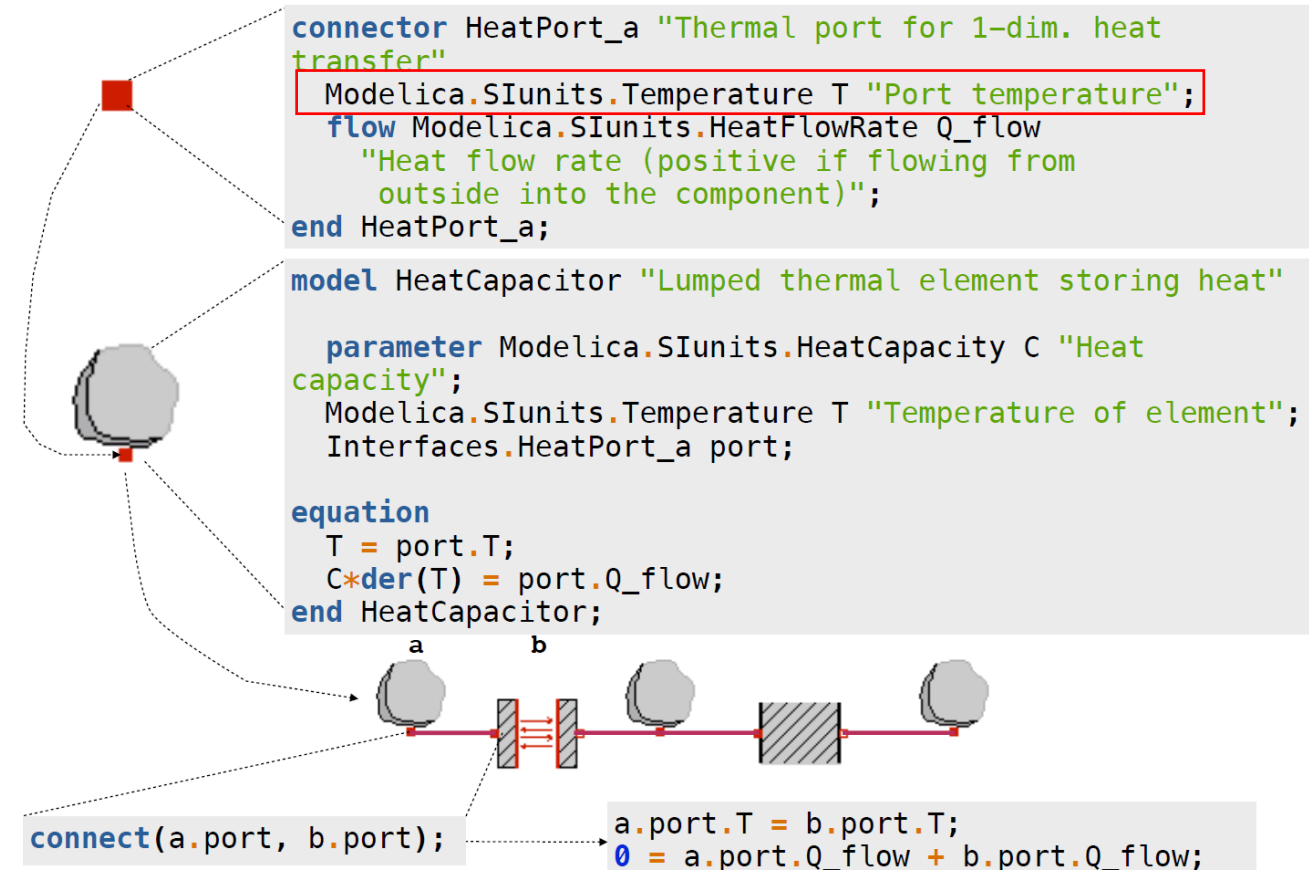


# Introduction to Modelica: Part II

## What sets Modelica apart?

### Physical modelling language

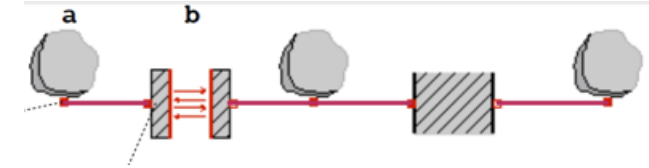
- Physics-based equation modeling
- Covering all physical domains
- Accurate bookkeeping of variable types and units
- Physical information passed through connectors.



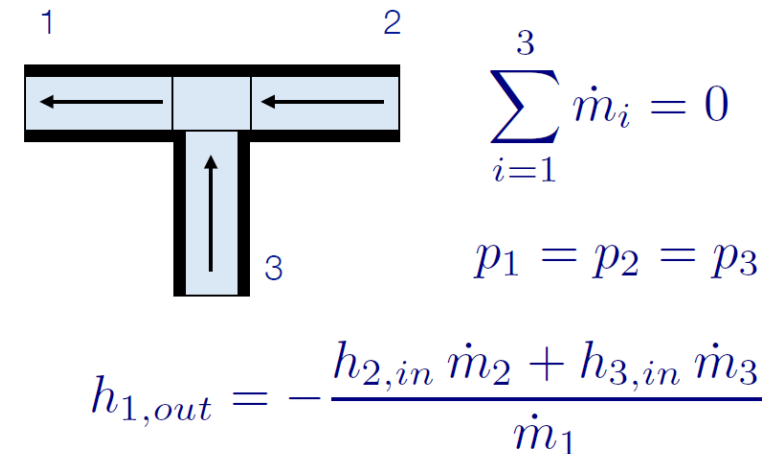
# Introduction to Modelica: Part II

## What sets Modelica apart?

- Efficient modelling: acausal connections
  - Input/output determined at compilation time
  - Conservation of physical principals (energy and mass balance etc.)
  - No. of equations = no. of variables at each level of model hierarchy



| Physical Domain | Potential (Across)   | Flow (Through)    | Stream                                                |
|-----------------|----------------------|-------------------|-------------------------------------------------------|
| Heat Transfer   | Temperature ( $T$ )  | Heat Flow ( $Q$ ) |                                                       |
| Fluid Flow      | Pressure ( $P$ )     | Mass Flow ( $m$ ) | Enthalpy ( $h$ )<br>Moisture ( $X$ )<br>Trace ( $C$ ) |
| Electrical      | Voltage ( $V$ )      | Current ( $I$ )   |                                                       |
| Translational   | Displacement ( $s$ ) | Force ( $F$ )     |                                                       |
| Rotational      | Angle ( $\theta$ )   | Torque ( $\tau$ ) |                                                       |

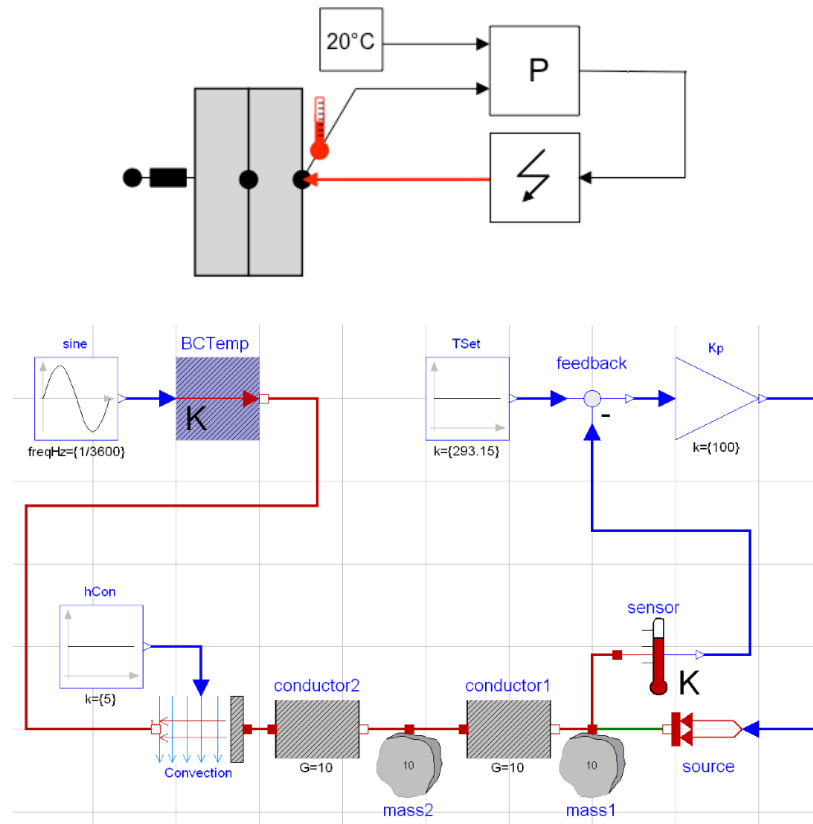




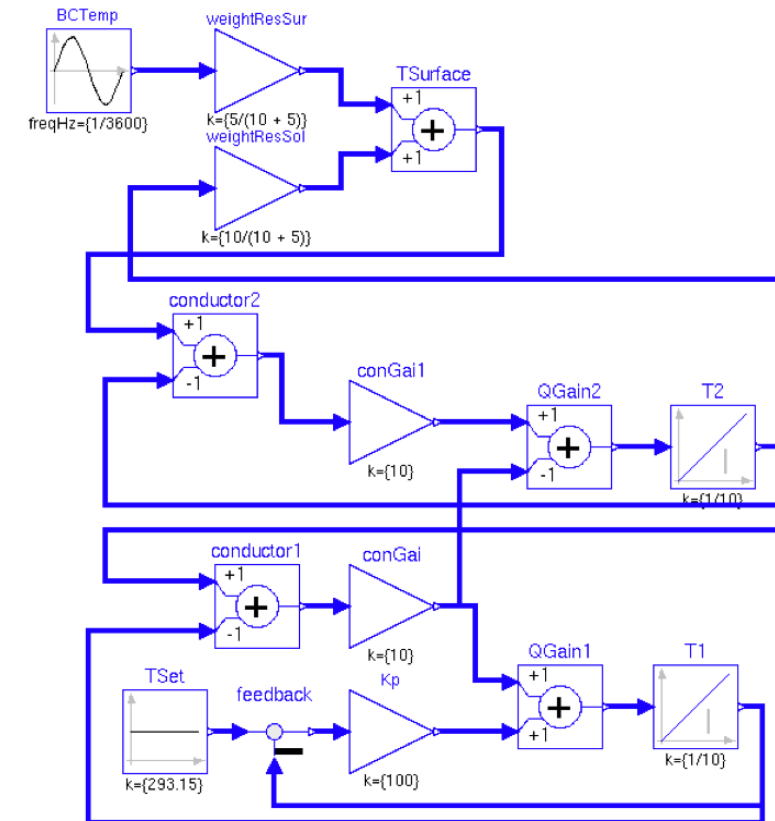
# Introduction to Modelica: Part II

## What sets Modelica apart?

- Efficient modelling: acausal connections
  - Component diagram leads to much higher readability and reusability



Component Diagram



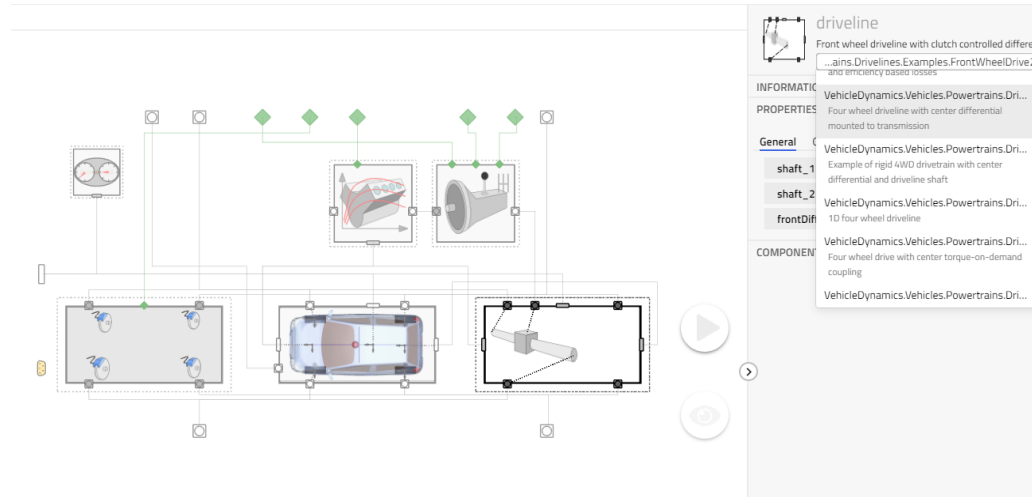
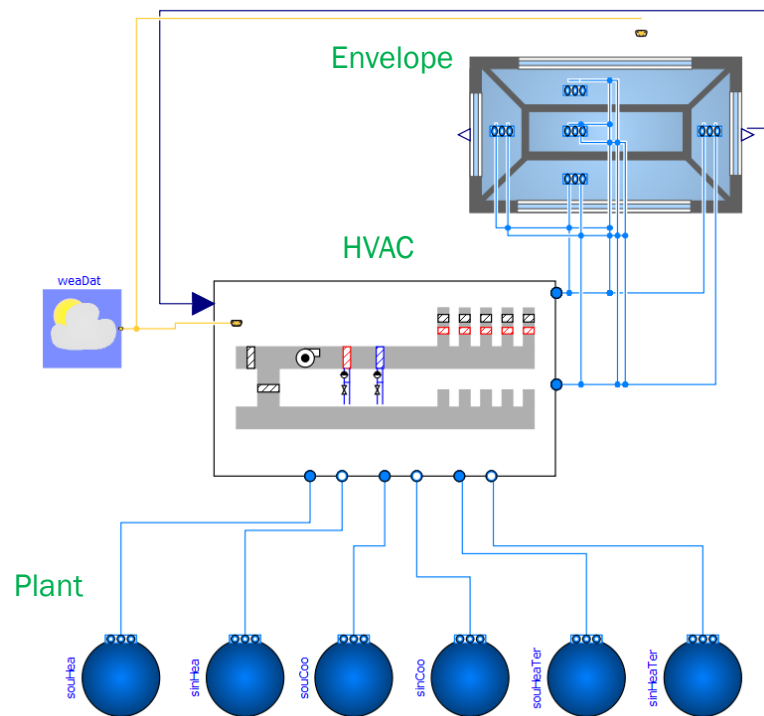
Block Diagram



# Introduction to Modelica: Part II

## What sets Modelica apart?

- Efficient modelling: architecture-based
  - Clearer representation and easier model maintenance
  - Handling different levels of fidelity and variations

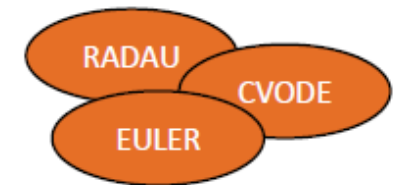




# Introduction to Modelica: Part II

## What sets Modelica apart?

- An integrated graphical and code interface
  - Build systems from graphical components (through drag & drop or templates)
  - Review the code and change to your needs
  - Documentation embedded in models
- Separation of models and solvers
  - DAE solvers appropriate for multi-physics and stiff systems
  - Advances in solver development can be leveraged
  - Variable timestep and fixed timestep





# Introduction to Modelica: Part II

## What sets Modelica apart?

### Modelica

- Integration of development and simulation environment: higher requirement for users, especially those who are afraid of programming
- Lack of users from building industry
- Translation, compilation and variable timestep integrator: longer simulation time
- Separation of models and solvers: users are exposed to errors related to solvers

### Existing mainstream BPS tools (DOE-2, EnergyPlus, TRNSYS, ESP-r, etc.)

- Developers vs. users
- Legacy code of existing BPS tools difficult to maintain and update, error-prone
- Lack of software architecture (DOE-2, E+)
- Unable to model dynamics under 1 min
- Unable to model control logic explicitly

### Simulink

- Lack of blocks for thermodynamics
- No access to source code



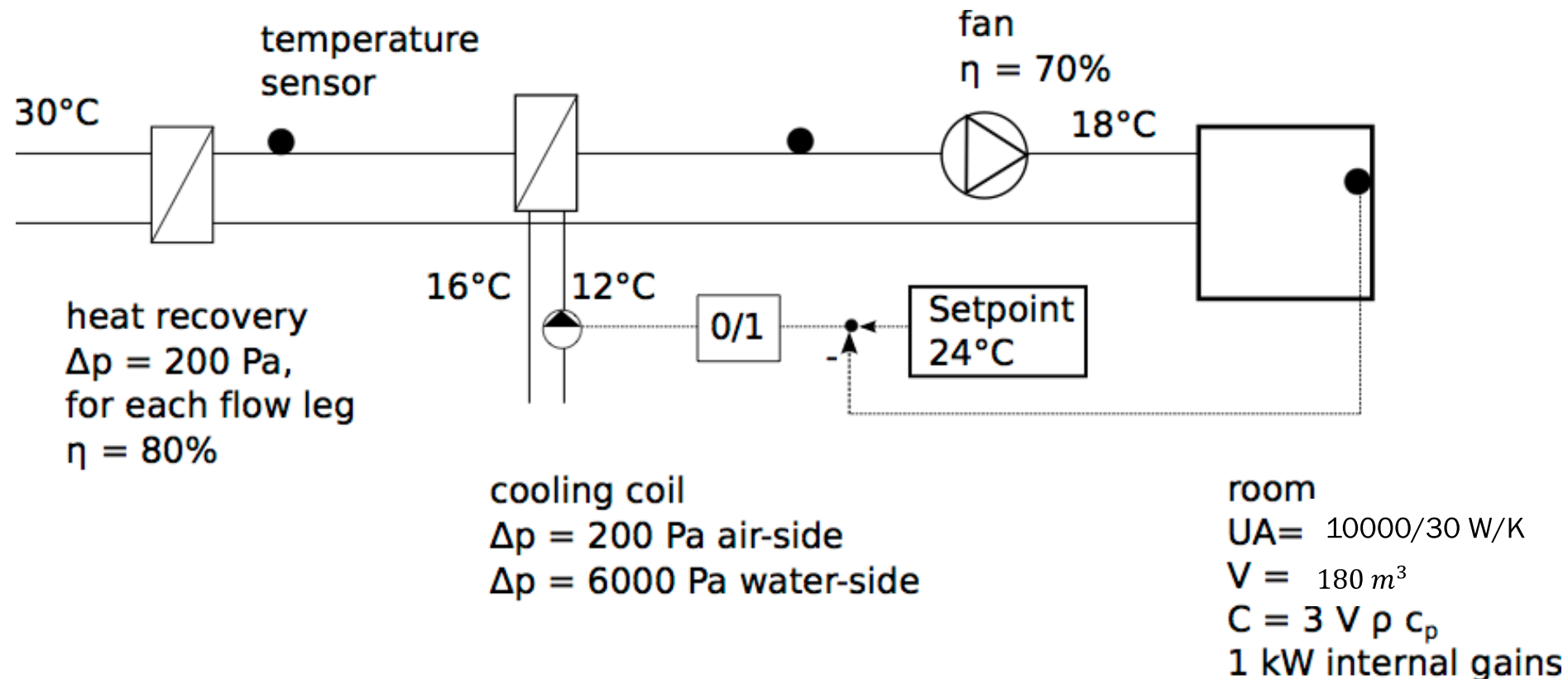
# Workshop plan

- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- **Tutorial Part II & Part III: Model construction using graphical interface**
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



# Model plan and workflow

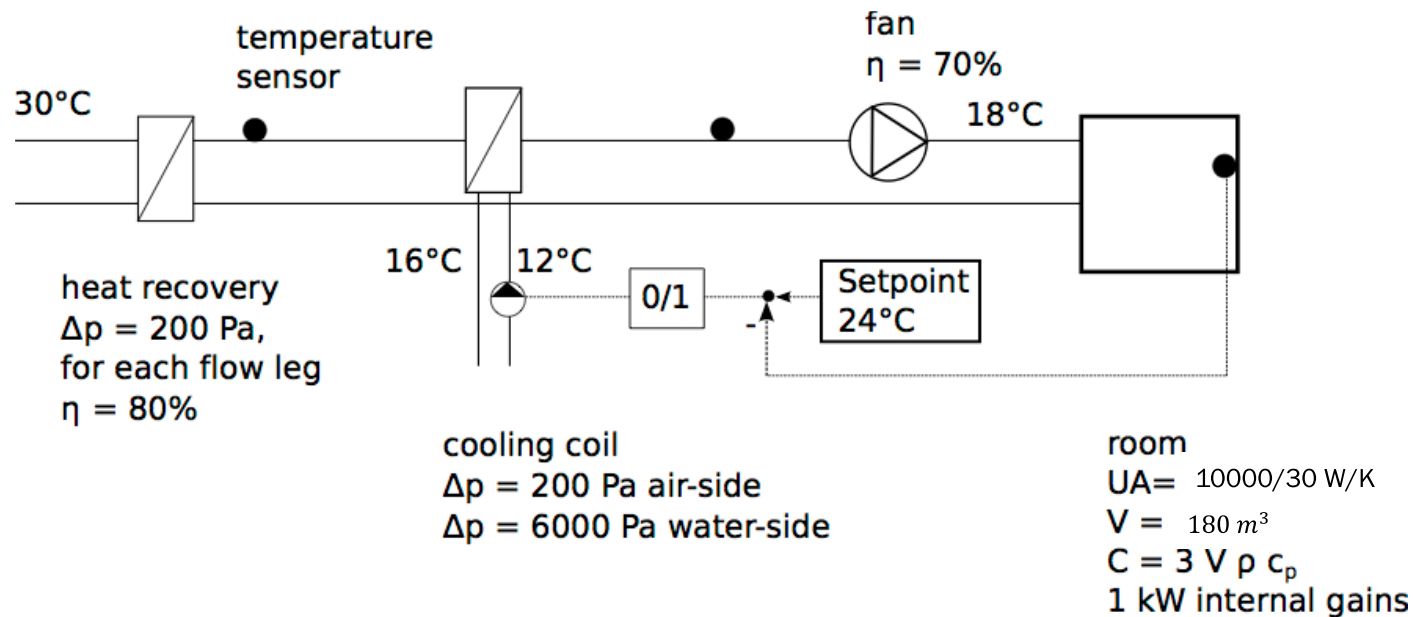
- Tutorial Part I: Modelling a room with 1R1C (Model 1)
- Tutorial Part II: Modelling an Air Handling Unit (AHU) (Model 2)
- Tutorial Part III: Modelling on/off control of an AHU (Model 3)





# Tutorial Part II: Model construction using graphical interface

## Model 2: Modeling an Air Handling Unit (AHU)



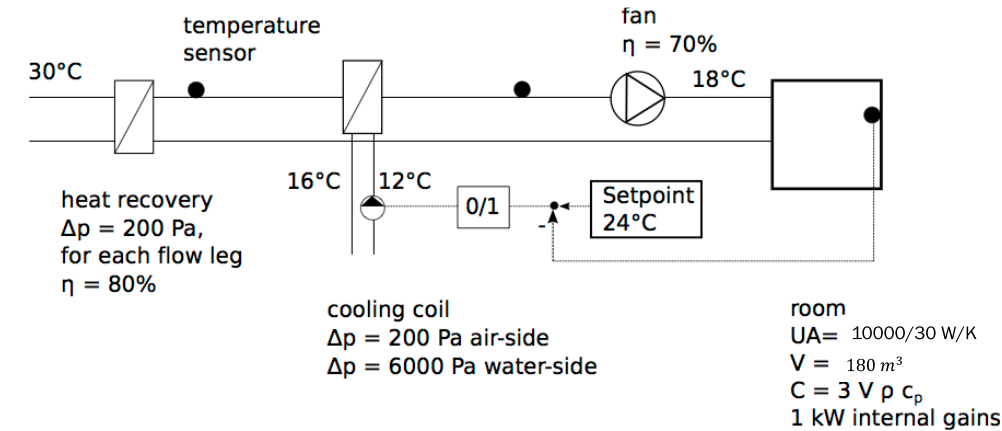
- AHU system
  - Composed of a heat recovery device, a cooling coil, a fan and two temperature sensors
- Design/nominal condition : need to be calculated/obtained first
  - Room temperature setpoint
  - Room cooling load
  - Supply air temperature
  - Supply air humidity
  - Nominal air flow rate
  - Chilled water supply temperature
  - Nominal water flow rate
  - Design capacity of cooling coils
  - .....



# Tutorial Part II: Model construction using graphical interface

## Model 2: Modeling an Air Handling Unit (AHU)

- Modelica components (airside)
  - Outside air: Buildings.Fluid.Sources.Boundary\_pT (**outAir**)
  - Heat recovery: Buildings.Fluid.HeatExchangers.ConstantEffectiveness (**hex**)
  - Temperature sensor (2): Buildings.Fluid.Sensors.TemperatureTwoPort (**senTem**, **senTem1**)
  - Cooling coil: Buildings.Fluid.HeatExchangers.WetCoilEffectivenessNTU (**cooCoil**)
  - Fan: Buildings.Fluid.Movers.FlowControlled\_m\_flow (**fan**)
  - Air flow control: Buildings.Controls.OBC.CDL.Reals.Sources.Constant (**flowCtrl**)
- Modelica components (waterside)
  - Chilled water supply: Buildings.Fluid.Sources.MassFlowSource\_T (**chiWatSource**)
  - Chilled water return: Buildings.Fluid.Sources.Boundary\_pT (**chiWatSink**)

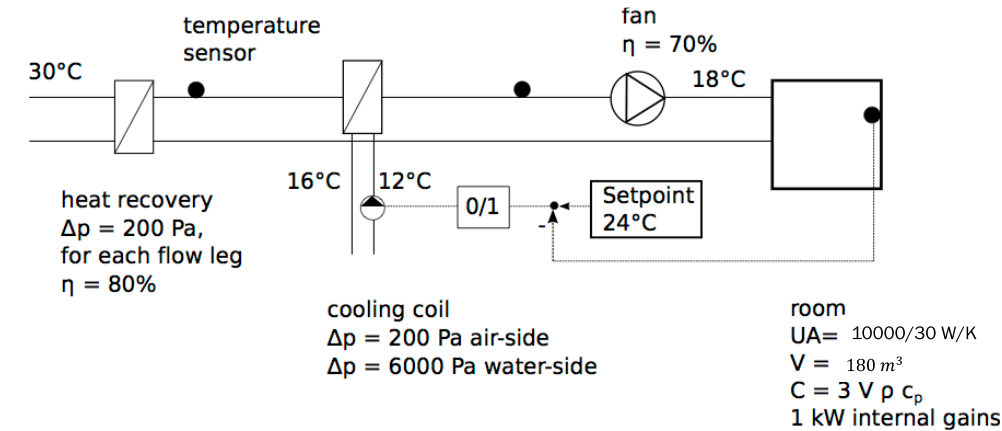




# Tutorial Part II: Model construction using graphical interface

## Model 2: Modeling an Air Handling Unit (AHU)

- Outside air:
  - Medium = Buildings.Media.Air "Moist air"
  - use\_T\_in = true
- Heat recovery:
  - Medium1 = Buildings.Media.Air "Moist air"
  - Medium2 = Buildings.Media.Air "Moist air"
  - m1\_flow\_nominal = 0.646 kg/s
  - m2\_flow\_nominal = 0.646 kg/s
  - dp1\_nominal = 200 Pa
  - dp2\_nominal = 200 Pa
- Temperature sensor (2):
  - Medium = Buildings.Media.Air "Moist air"
  - m\_flow\_nominal = 0.646 kg/s



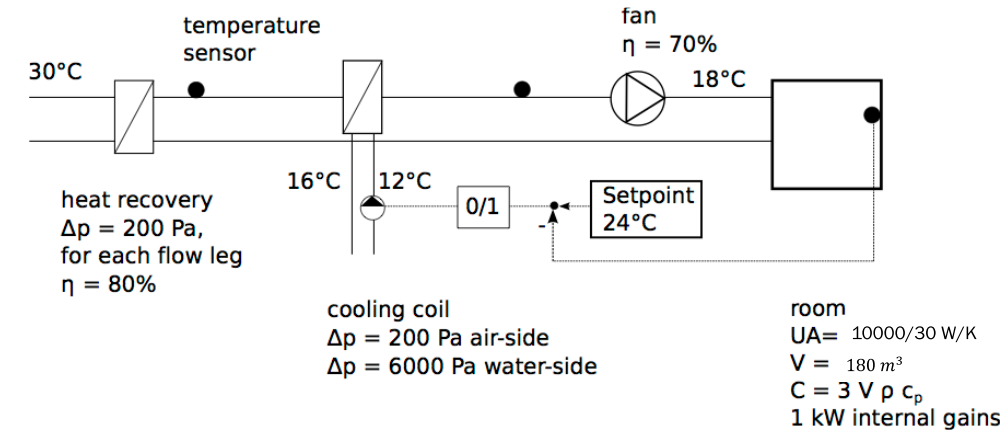


# Tutorial Part II: Model construction using graphical interface

## Model 2: Modeling an Air Handling Unit (AHU)

- Cooling coil:

- Medium 1 = Buildings.Media.Water "Water"
- Medium 2 = Buildings.Media.Air "Moist air"
- $m1\_flow\_nominal = 0.498 \text{ kg/s}$  (chilled water flow)
- $m2\_flow\_nominal = 0.646 \text{ kg/s}$  (cooling air flow rate)
- $dp1\_nominal = 6000 \text{ Pa}$
- $dp2\_nominal = 200 \text{ Pa}$
- $use\_Q\_flow\_nominal = \text{True}$
- $Q\_flow\_nominal = -8362 \text{ W}$
- $T_{a1\_nominal} = 12^\circ\text{C}$  (chilled water inlet temperature)
- $T_{a2\_nominal} = 25.2^\circ\text{C}$  (air inlet temperature)
- $w_{a2\_nominal} = 0.0135 \text{ kg/kg}$  (air inlet humidity ratio)
- `energyDynamics = Modelica.Fluid.Types.Dynamics.FixedInitial`  
(see **tab: Dynamics**)

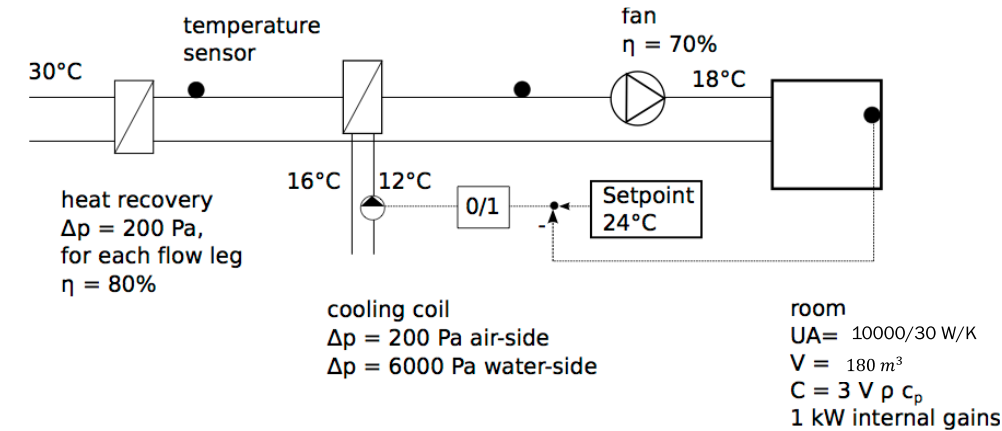




# Tutorial Part II: Model construction using graphical interface

## Model 2: Modeling an Air Handling Unit (AHU)

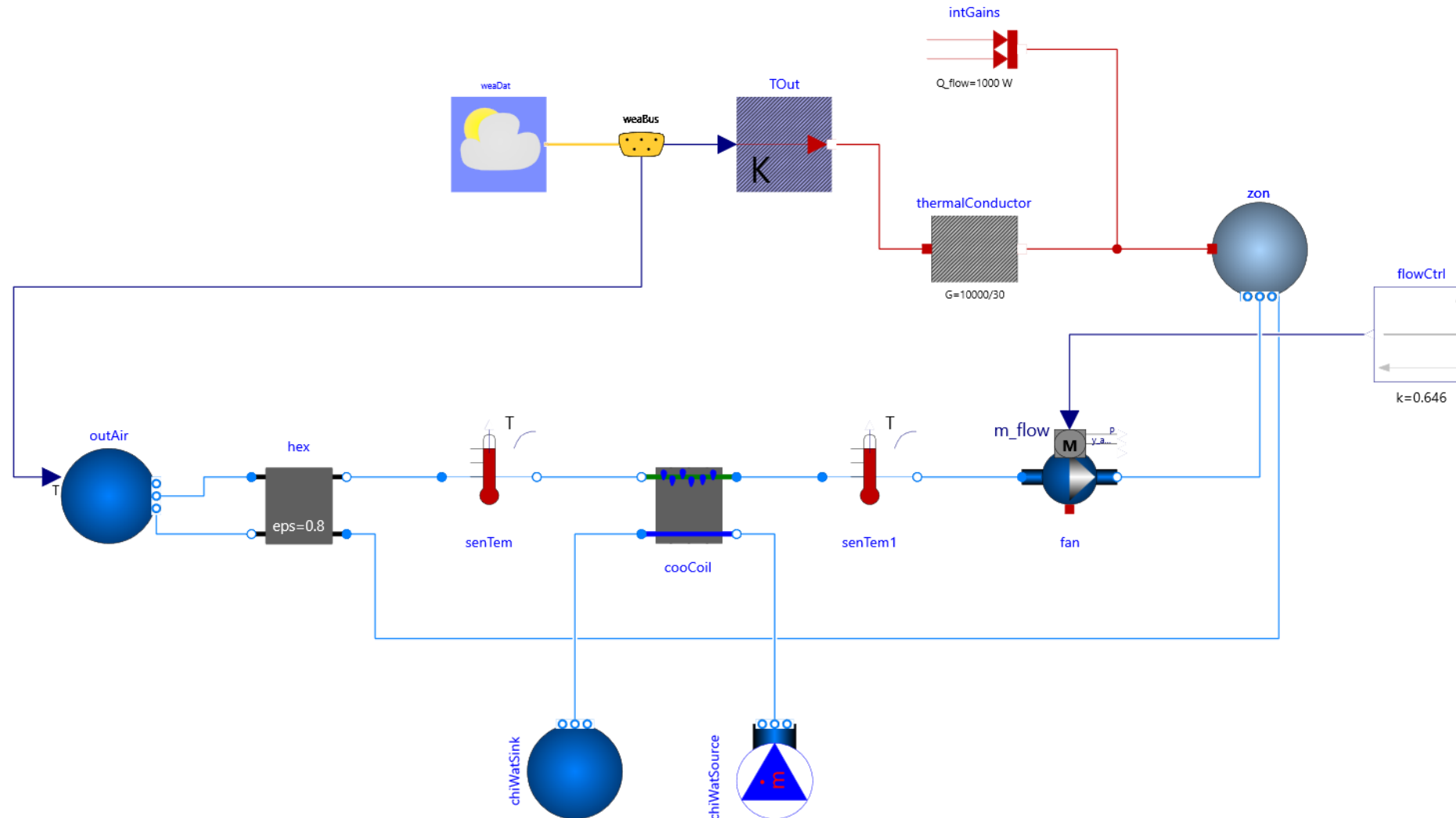
- Fan
  - Medium = Buildings.Media.Air "Moist air"
  - $m\_flow\_nominal = 0.646 \text{ kg/s}$
  - $energyDynamics = Modelica.Fluid.Types.Dynamics.SteadyState$  (see **tab: Dynamics**)
- flowCtrl
  - $k = 0.646$
  - Flip Horizontal
- Water source:
  - Medium = Buildings.Media.Water "Water"
  - $m\_flow = 0.498 \text{ kg/s}$
  - $T = 12^\circ\text{C}$
- Connect all components





# Tutorial Part II: Model construction using graphical interface

## Model 2: Modeling an Air Handling Unit (AHU)





# Tutorial Part II: Model construction using graphical interface

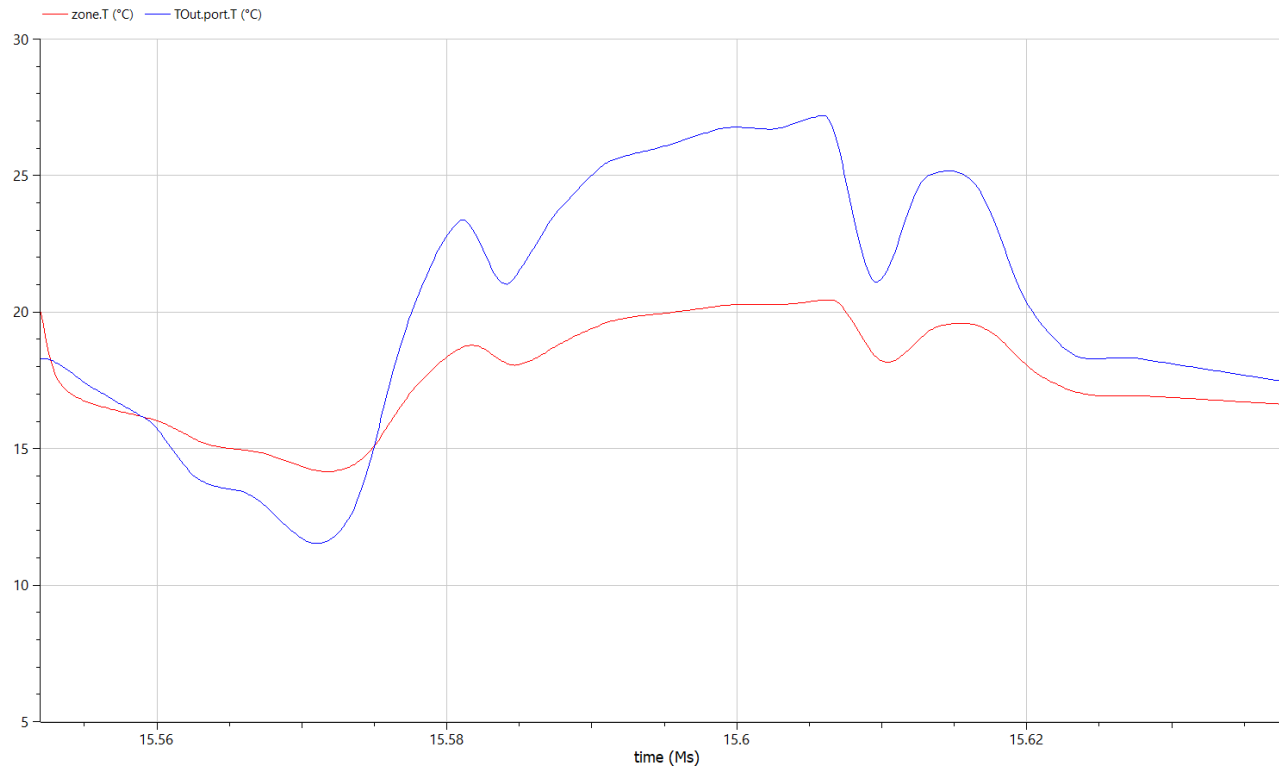
## Model 2: Modeling an Air Handling Unit (AHU)

Simulation setup:

Start time: 180 d (June 30) = 4320 h = 15552000 s

Stop time: 181 d = 4344 h = 15638400 s

Tick box: save experiment annotation



### Simulation Setup - Room\_cooling

General Interactive Simulation Translation Flags Simulation Flags Output Linearize

Simulation Interval

Start Time: 15552000 secs

Stop Time: 15638400 secs

☒ Number of Intervals: 500

☐ Interval: 172.8 secs

Integration

Method: dassl

Tolerance: 1e-06

Jacobian:

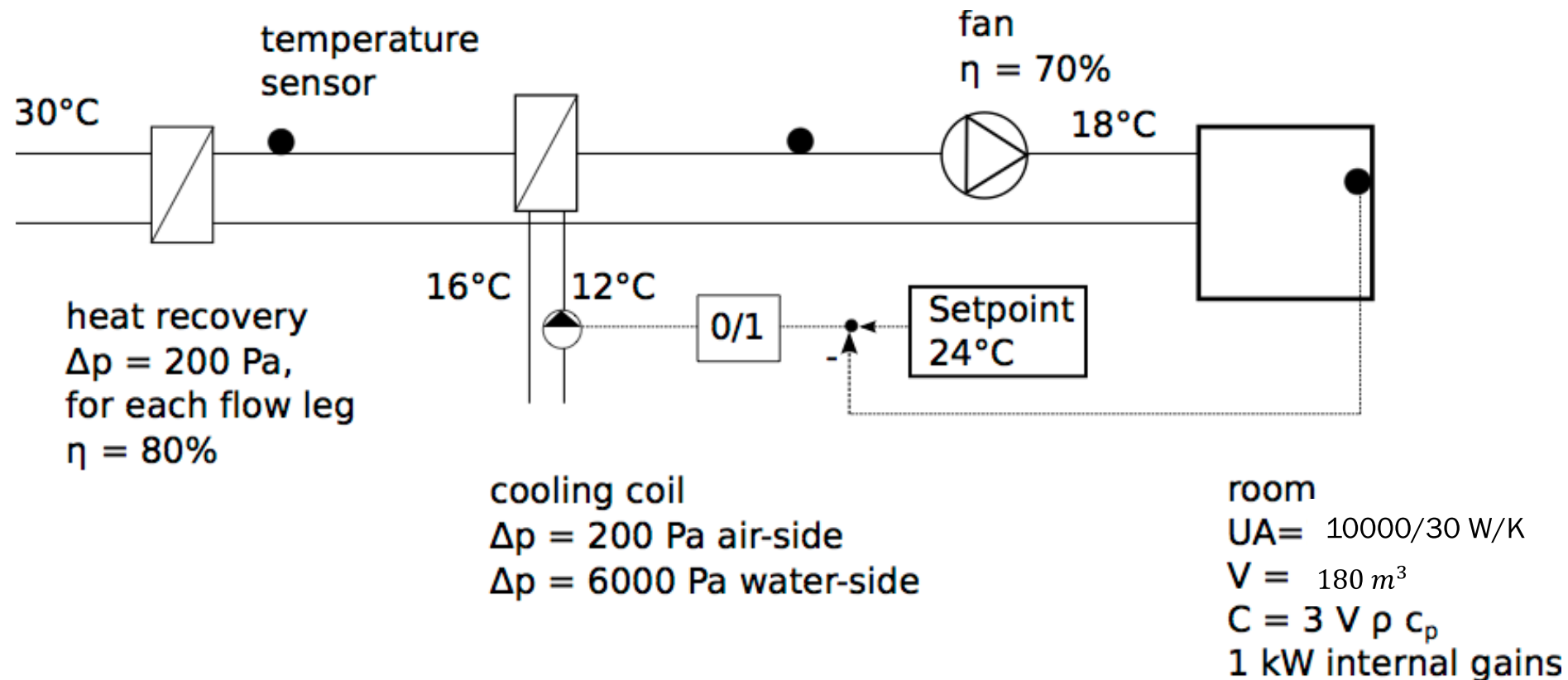
Options

☒ Save experiment annotation inside model i.e., experiment annotation



# Model plan and workflow

- Tutorial Part I: Modelling a room with 1R1C (Model 1)
- Tutorial Part II: Modelling an Air Handling Unit (AHU) (Model 2)
- Tutorial Part III: Modelling on/off control of an AHU (**Model 3**)

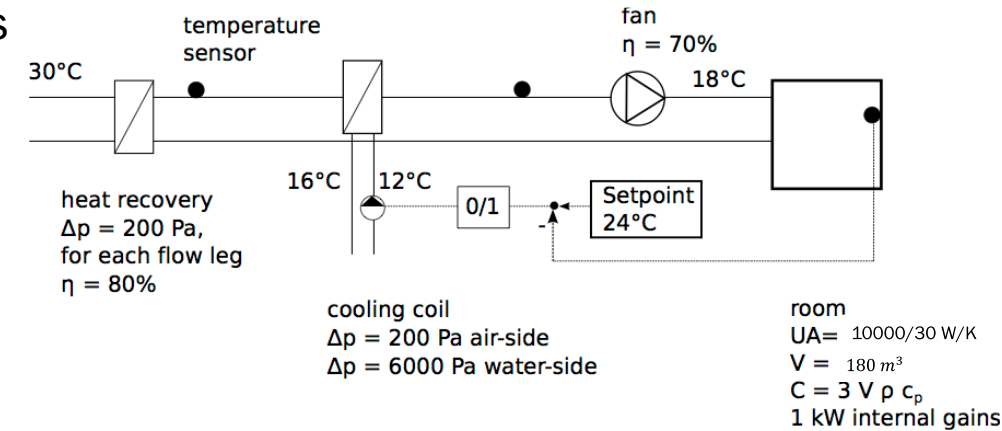




# Tutorial Part III: Model construction using graphical interface

## Model 3: Modelling on/off control of an AHU

- Control logic
  - Control chilled water flowrate based on room temperature and its setpoint
- Controller components
  - Temperature sensor, controller
- Modelica control components
  - Room temperature sensor: `Modelica.Thermal.HeatTransfer.Sensors.TemperatureSensor` (**zonTemSen**)
  - On/off controller: `Buildings.Controls.OBC.CDL.Reals.Hysteresis` (**controller**)
  - Boolean to Real: `Buildings.Controls.OBC.CDL.Conversions.BooleanToReal` (**booToRea**)
- Modelica components (waterside)
  - Chilled water supply: `Buildings.Fluid.Sources.MassFlowSource_T`
    - Set the parameter `use_m_flow_in` = True

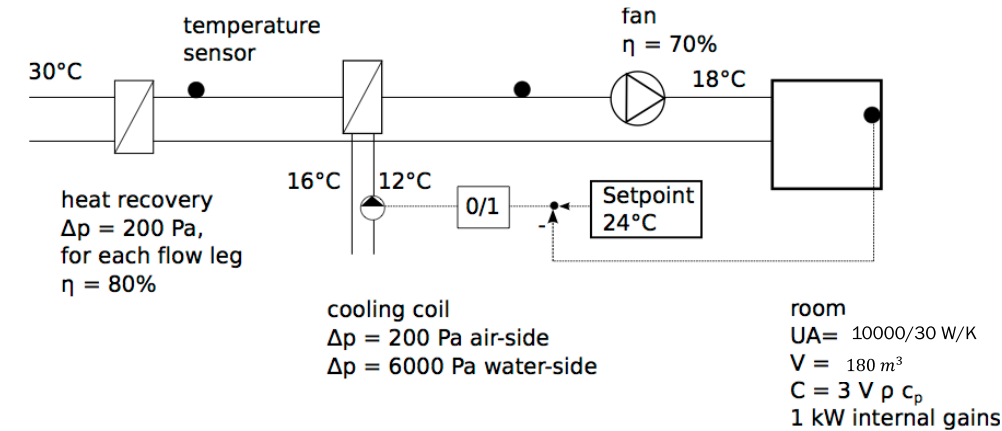




# Tutorial Part III: Model construction using graphical interface

## Model 3: Modelling on/off control of an AHU

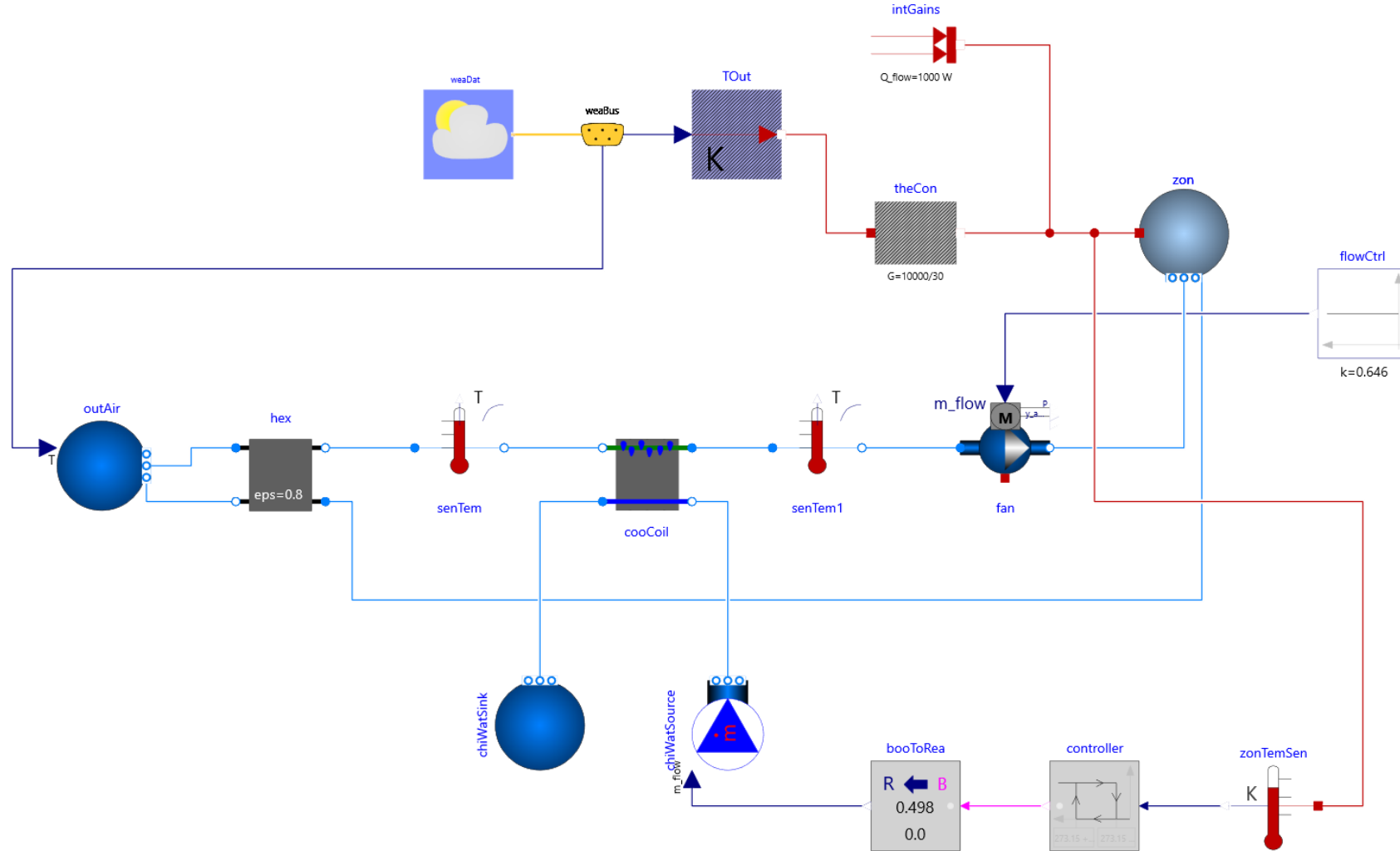
- On/off controller:  
Buildings.Controls.OBC.CDL.Reals.Hysteresis
  - 24 °C with 1 °C deadband
  - $u_{Low} = 273.15 + 23.5$ ;
  - $u_{High} = 273.15 + 24.5$
- Boolean to Real:  
Buildings.Controls.OBC.CDL.Conversions.BooleanToReal
  - $realTrue = 0.498$ ;
  - $realFalse = 0$
- Connect components
  - Flip Horizontal





# Tutorial Part III: Model construction using graphical interface

## Model 3: Modelling on/off control of an AHU





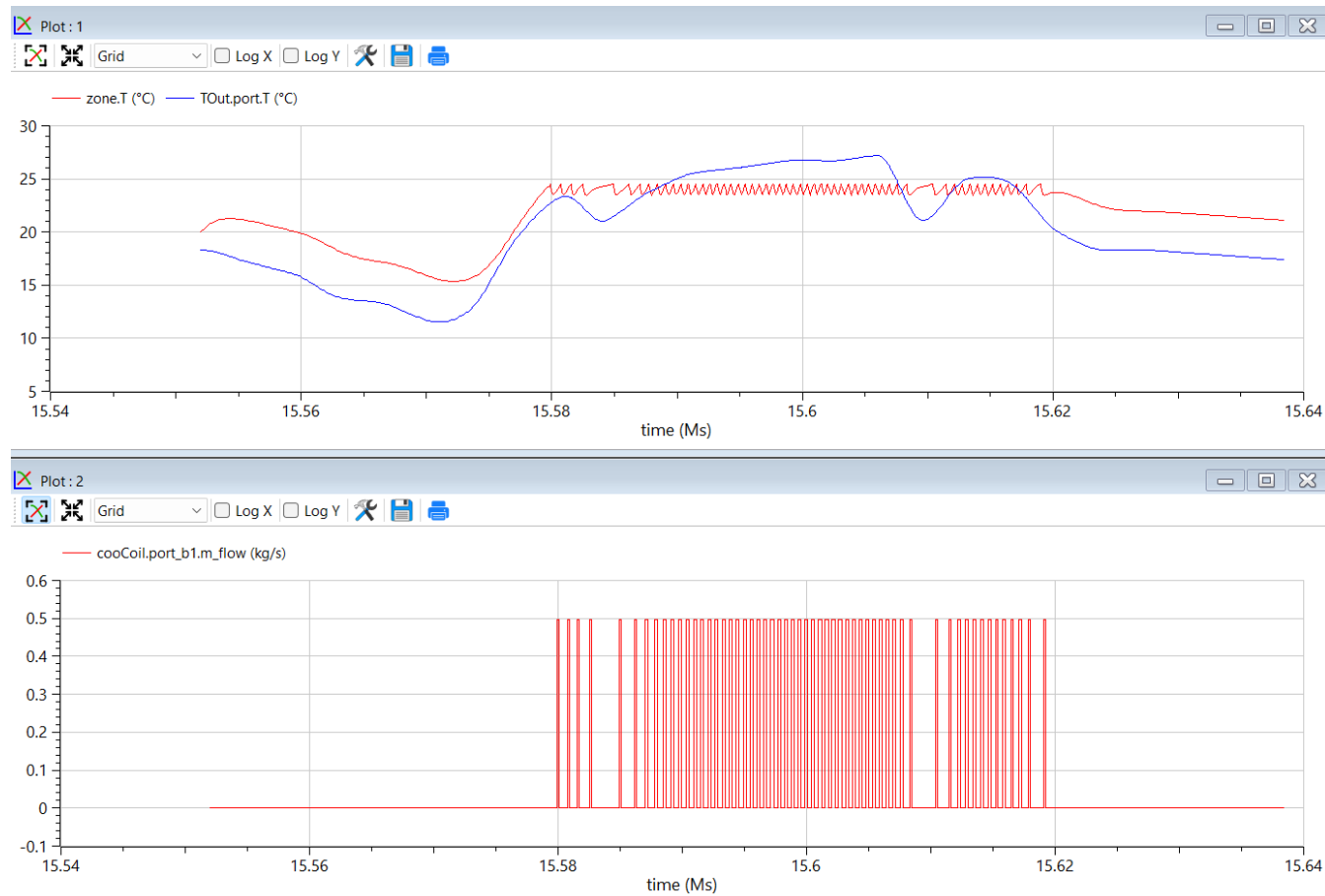
# Tutorial Part III: Model construction using graphical interface

## Model 3: Modelling on/off control of an AHU

Simulation setup:

Start time: 180 d (June 30) = 4320 h = 15552000 s

Stop time: 181 d = 4344 h = 15638400 s





# Workshop plan

- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



# Modelica projects

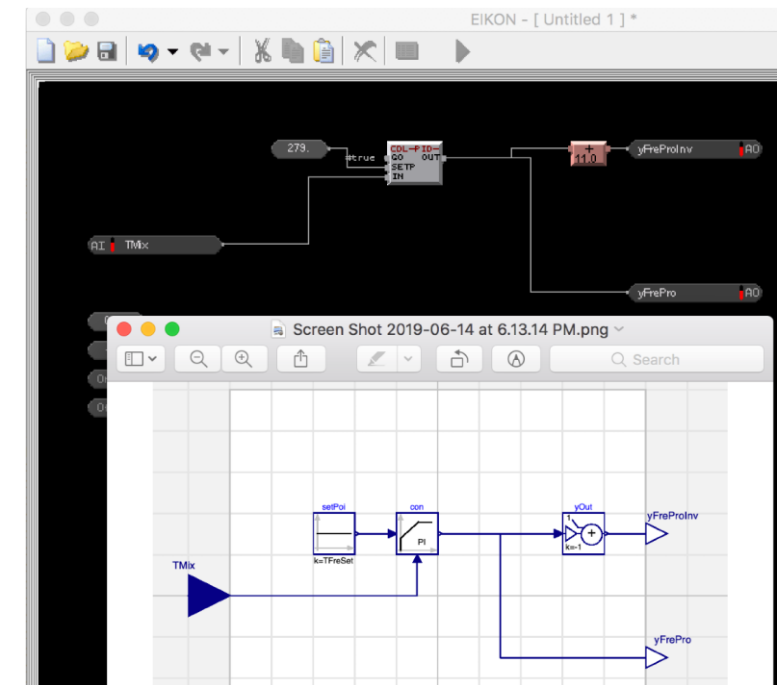
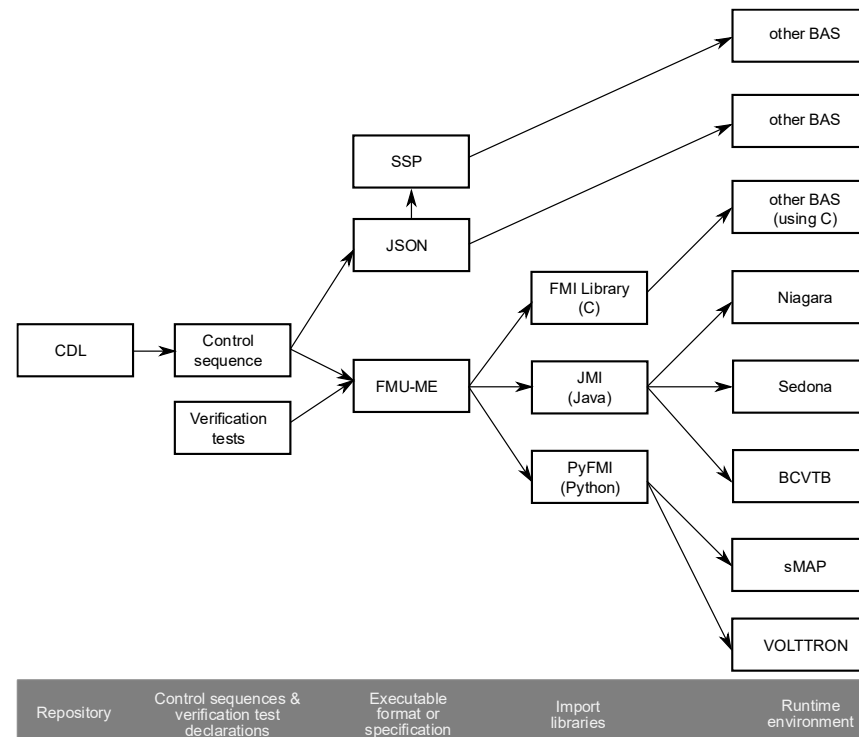
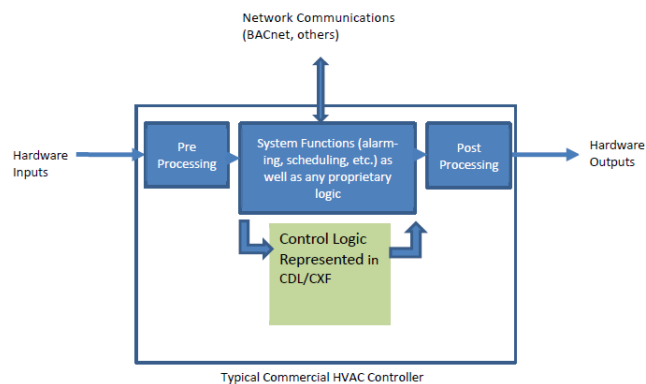
## Control Description Language (CDL)

- Subset of Modelica elementary blocks with adaptations and additions
  - ASHRAE Standard 231-2026: A Control Description Language for Building Environmental Control Sequences



**A Control Description Language for Building Environmental Control Sequences**

| Name                                                                                                  | Description                                          |
|-------------------------------------------------------------------------------------------------------|------------------------------------------------------|
|  <u>Constants</u>      | Package with constants                               |
|  <u>Conversions</u>    | Package with blocks for type conversion              |
|  <u>Discrete</u>       | Package with discrete blocks                         |
|  <u>Integers</u>       | Package with blocks for integer variables            |
|  <u>Logical</u>        | Package with logical blocks                          |
|  <u>Psychrometrics</u> | Package with psychrometric blocks                    |
|  <u>Reals</u>          | Package with blocks for continuous variables         |
|  <u>Routing</u>        | Package with blocks that combine and extract signals |
|  <u>Utilities</u>      | Package with utility functions                       |
|  <u>Types</u>          | Package with type definitions                        |
|  <u>Interfaces</u>     | Package with connectors for input and output signals |



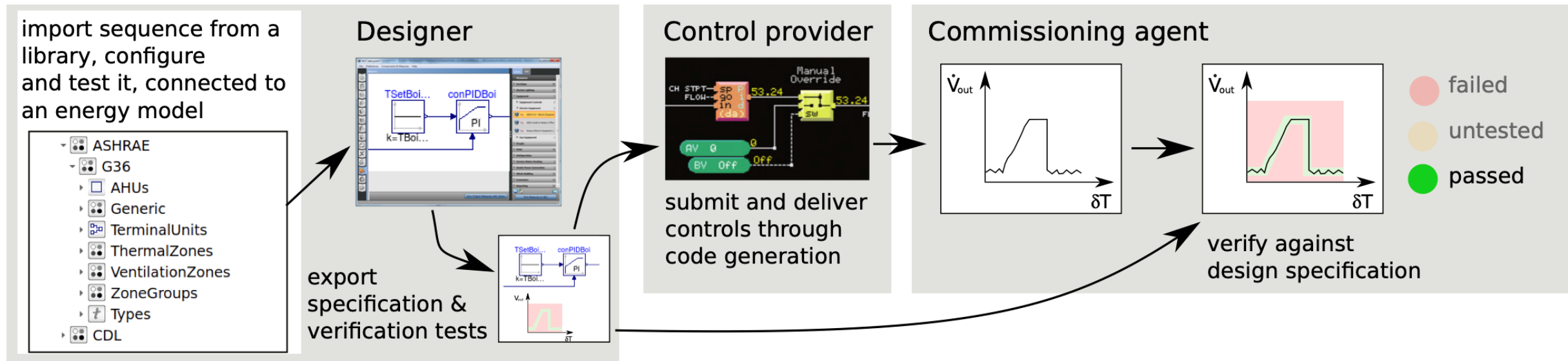
CDL blocks in “Eikon” (Automated Logic)



# Modelica projects

## Open Building Control

- Performance Evaluation, Specification, Deployment and Verification of Building Control Sequences
  - <https://obc.lbl.gov/>

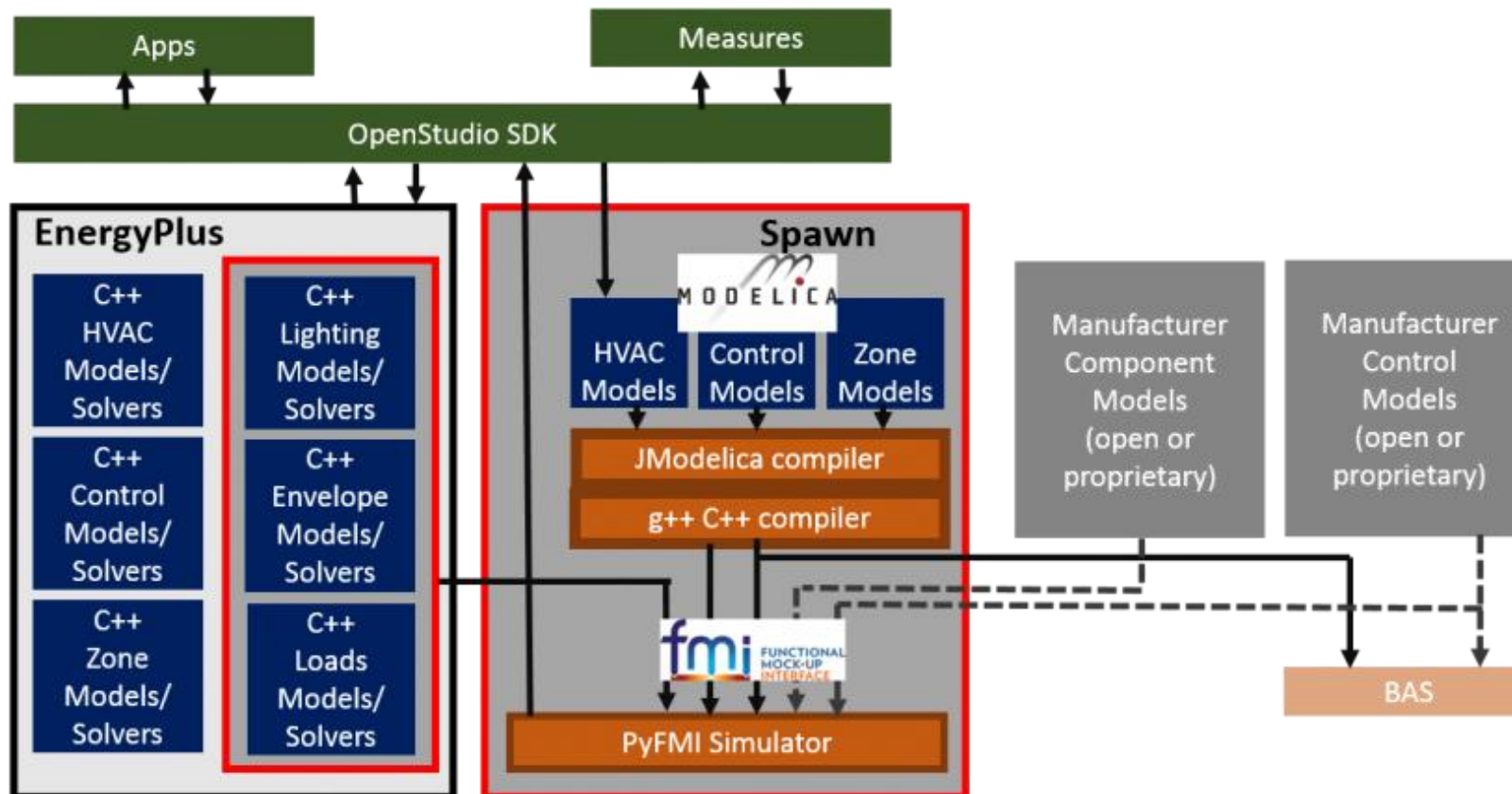




# Modelica projects

## Spawn of EnergyPlus

- Next-generation building energy modelling (BEM) engine
  - <https://lbl-srg.github.io/soep/index.html>

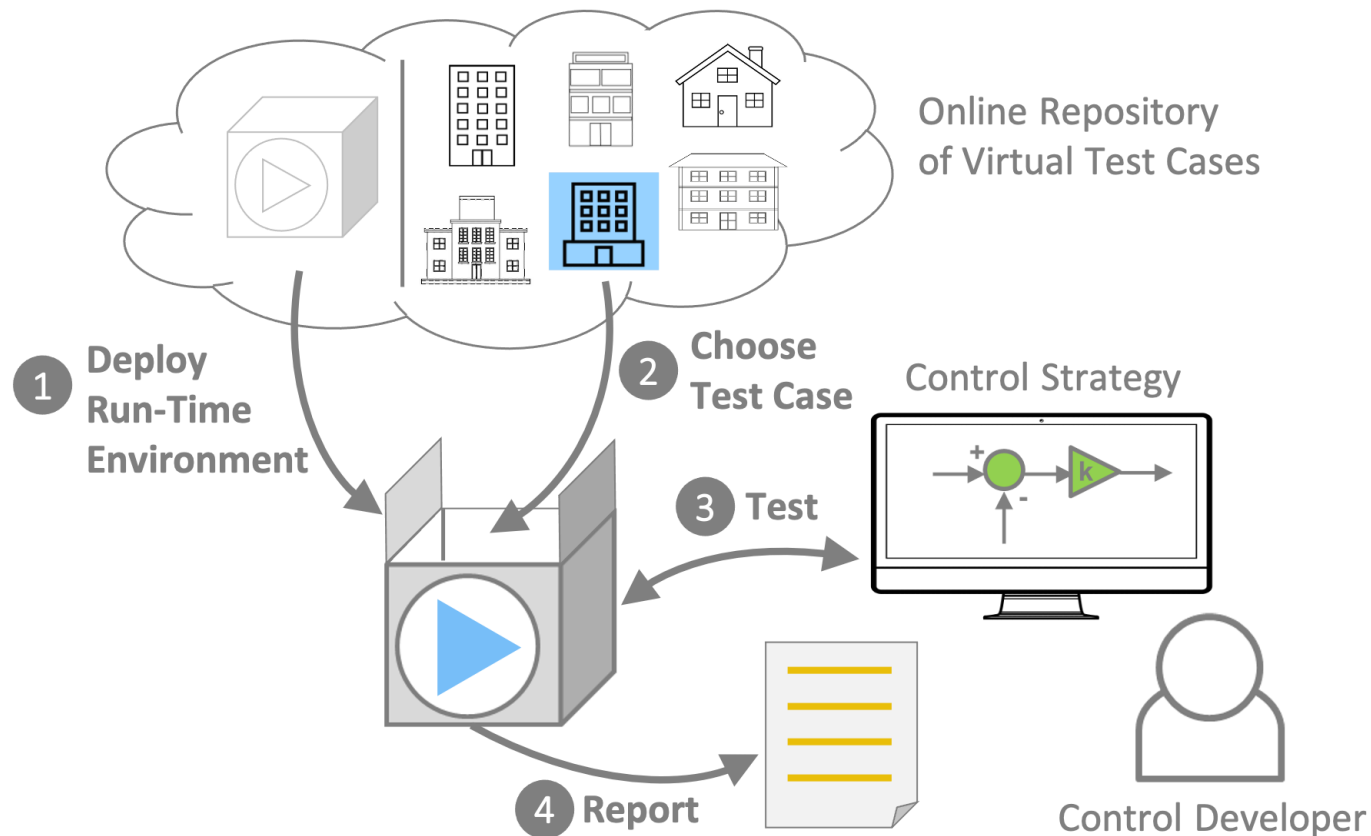




# Modelica projects

## BOPTEST (Building Optimization Testing Framework)

- A framework for building control performance benchmarking
  - IBPSA Project 2: <https://ibpsa.github.io/project1-boptest/>



AVAILABLE IN LATEST RELEASE (V0.9.0)

BESTEST Air  
BESTEST Hydronic  
BESTEST Hydronic Heat Pump  
Single Zone Commercial Hydronic  
Two Zone Apartment Hydronic  
Multizone Residential Hydronic  
Multizone Office Simple Air  
Multizone Office Simple Hydronic  
Multizone Office Complex Air

UNDER DEVELOPMENT

Single Zone Commercial Air  
Flexible Research Platform



# Best practices: Isolation of sub-models

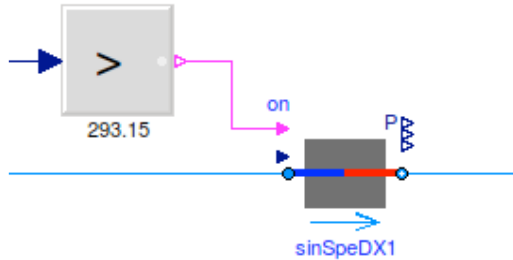
How do you build and debug a large system model?

- Understand the problem: what question do you want to answer?
- Know what you want to model: draw system schematics
- Split the model into smaller sub-models (e.g., room, HVAC, control)
- Test the sub-models with known conditions
  - Errors are easier to detect in small models, but harder in large models!
- Assemble the sub-models to build the full model



# Best practices: Adding hysteresis

Avoid this configuration



```
model HeatingControl
```

```
    Boolean heat "Indicates if heater is on";
```

```
equation
```

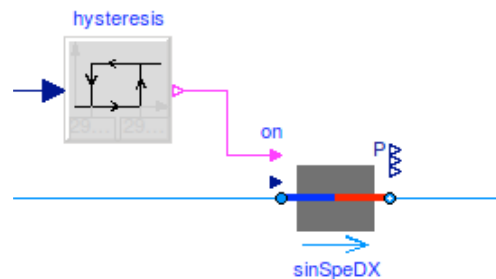
```
    heat = T<293.15;
```

```
    Q = if heat then Qcapacity else 0;
```

```
end HeatingControl;
```

- This works in fixed time step simulators, but won't in variable time step simulators that handle events.
- If the control error oscillates around zero, then this model stalls due to numerical noise and fast switching.

Correct configuration



```
model HeatingControl
```

```
    Boolean heat "Indicates if heater is on";
```

```
equation
```

```
    when {T<293.15-1, T>293.15+1} then
```

```
        heat = T<293.15;
```

```
    end when;
```

```
    Q = if heat then Qcapacity else 0;
```

```
end HeatingControl;
```

- The “when” condition is active when any element is true
- Represents a real-world on/off controller with a **dead band**



# Best practices: Parametrization

```
Pump pum(m_flow_nominal=0.1) "Pump";  
TemperatureSensor sen(m_flow_nominal=0.1) "Sensor";
```

```
Modelica.SIunits.MassFlowRate m_flow_nominal = 0.1  
"Nominal mass flow rate";  
Pump pum(final m_flow_nominal=m_flow_nominal) "Pump";  
TemperatureSensor sen(final m_flow_nominal=m_flow_nominal) "Sensor";
```

- Parameterization avoids repetition: same for all programming practice!
- Easier management: only needs to update once and avoids misses
- Works also for media package
- Refactoring Model 3

```
replaceable package MediumA = Buildings.Media.Air  
"Medium model";
```

```
TemperatureSensor sen(  
  redeclare final package Medium = MediumA,  
  final m_flow_nominal=m_flow_nominal) "Sensor";
```



# Wrap up

- Discussed the Modelica language and its differences from other tools
- Demonstrated on Modelica Standard Library and Buildings Library
- Presented Modelica projects
- Guided step-by-step on model construction
- Analyzed simulation results
- Developed a simple Modelica model using programming
- Learned best practices
- Built independently a Modelica model



# Self-learning: Useful online resources

- Modelica Association: <https://modelica.org/>
- Modelica by Example: <https://mbe.modelica.university/>
- Training materials for Modelica Buildings Library:  
<https://simulationresearch.lbl.gov/modelica/training.html>
- Modelica Buildings Library Users Guide:  
<https://simulationresearch.lbl.gov/modelica/userGuide/index.html>
- Coding style and convention:  
<https://simulationresearch.lbl.gov/modelica/userGuide/development.html>



# Workshop plan

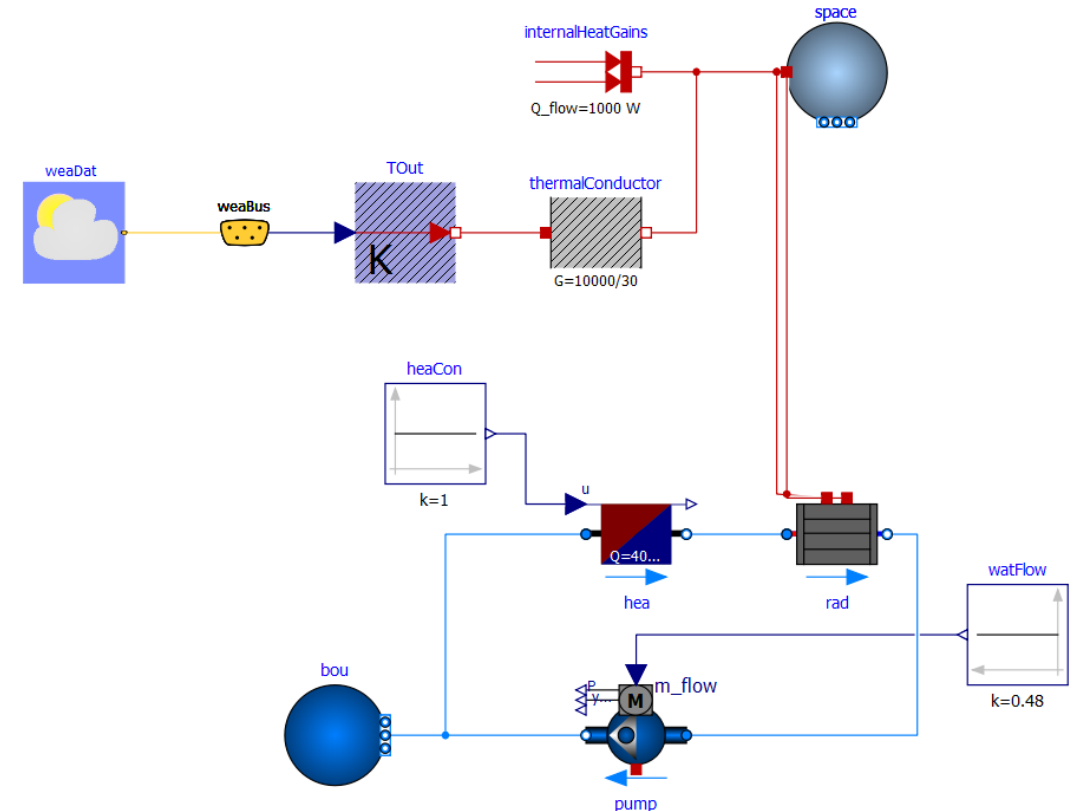
- Introduction to Modelica: Part I
  - Introduction to Modelica (the language)
  - Introduction to OpenModelica (the open-source software)
- Tutorial Part I:
  - Model construction using programming
  - Model construction using graphical interface
- Introduction to Modelica: Part II
- Tutorial Part II & Part III: Model construction using graphical interface
- Introduction to Modelica projects, best practices and wrap-up
  - Exercise on Modelica model construction



# Exercise on Modelica model construction

## Space heating with radiators

- Use the same room model
- Radiator:  
Buildings.Fluid.HeatExchangers.Radiators.RadiatorEN442\_2
  - $Q_{\text{flow\_nominal}} = 40 \text{ kW}$
  - $T_{\text{a\_nominal}} = 60 \text{ }^{\circ}\text{C}$ ;  $T_{\text{b\_nominal}} = 40 \text{ }^{\circ}\text{C}$
- Heater:  
Buildings.Fluid.HeatExchangers.HeaterCooler\_u
  - $Q_{\text{flow\_nominal}} = 40 \text{ kW}$
  - $m_{\text{flow\_nominal}} = 0.48 \text{ kg/s}$ ;  $dp_{\text{nominal}} = 2000 \text{ Pa}$
- Pump:  
Buildings.Fluid.Movers.FlowControlled\_m\_flow
- Water source and sink:  
Buildings.Fluid.Sources.Boundary\_pT





# Exercise on Modelica model construction

## Space heating with radiators and control

- Control logic
  - Control the heater based on room temperature and its setpoint
  - Control the water flow rate based on the same control signal
- On/off controller: 21 °C with 1K dead band
  - Buildings.Controls.OBC.CDL.Reals.Hysteresis

